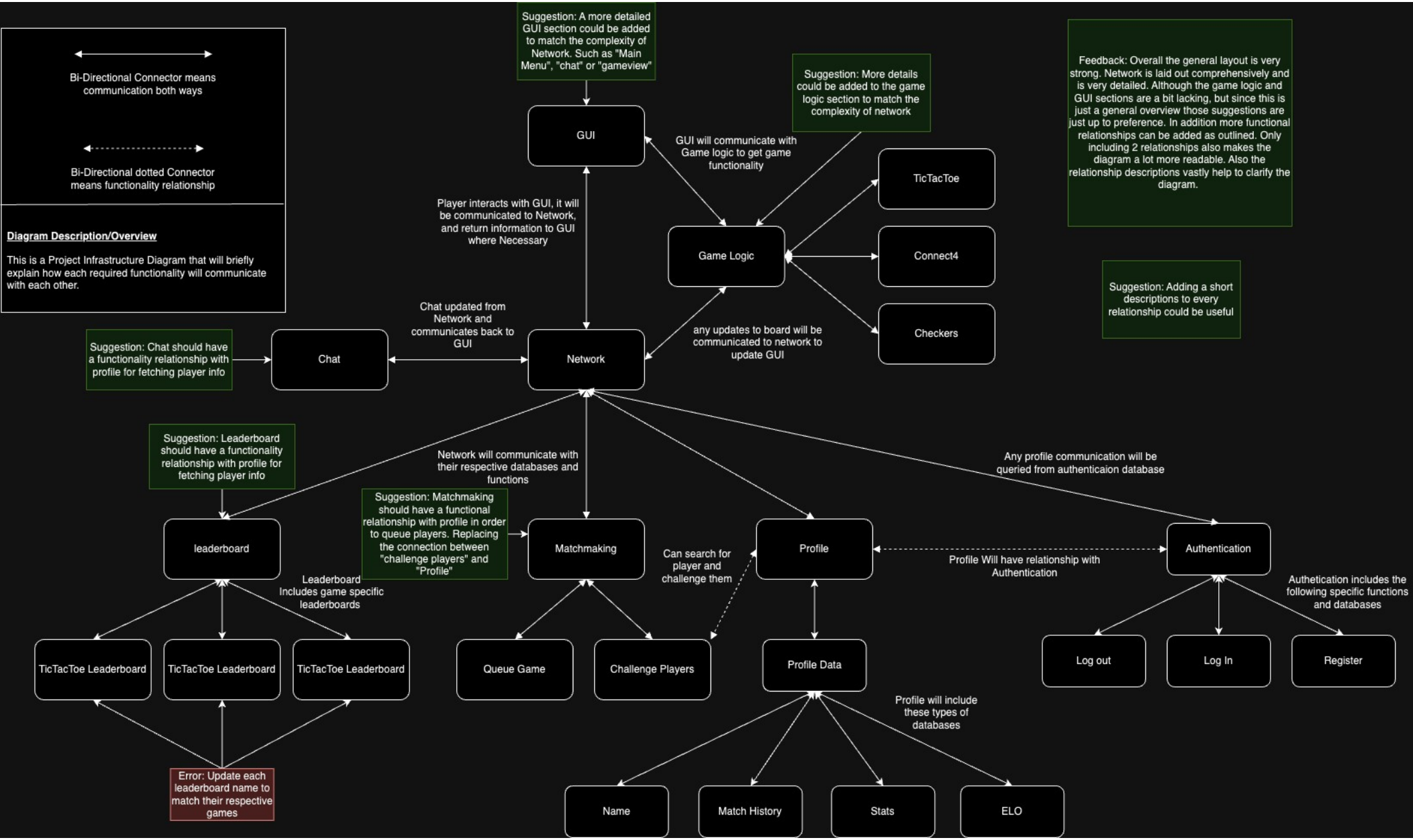
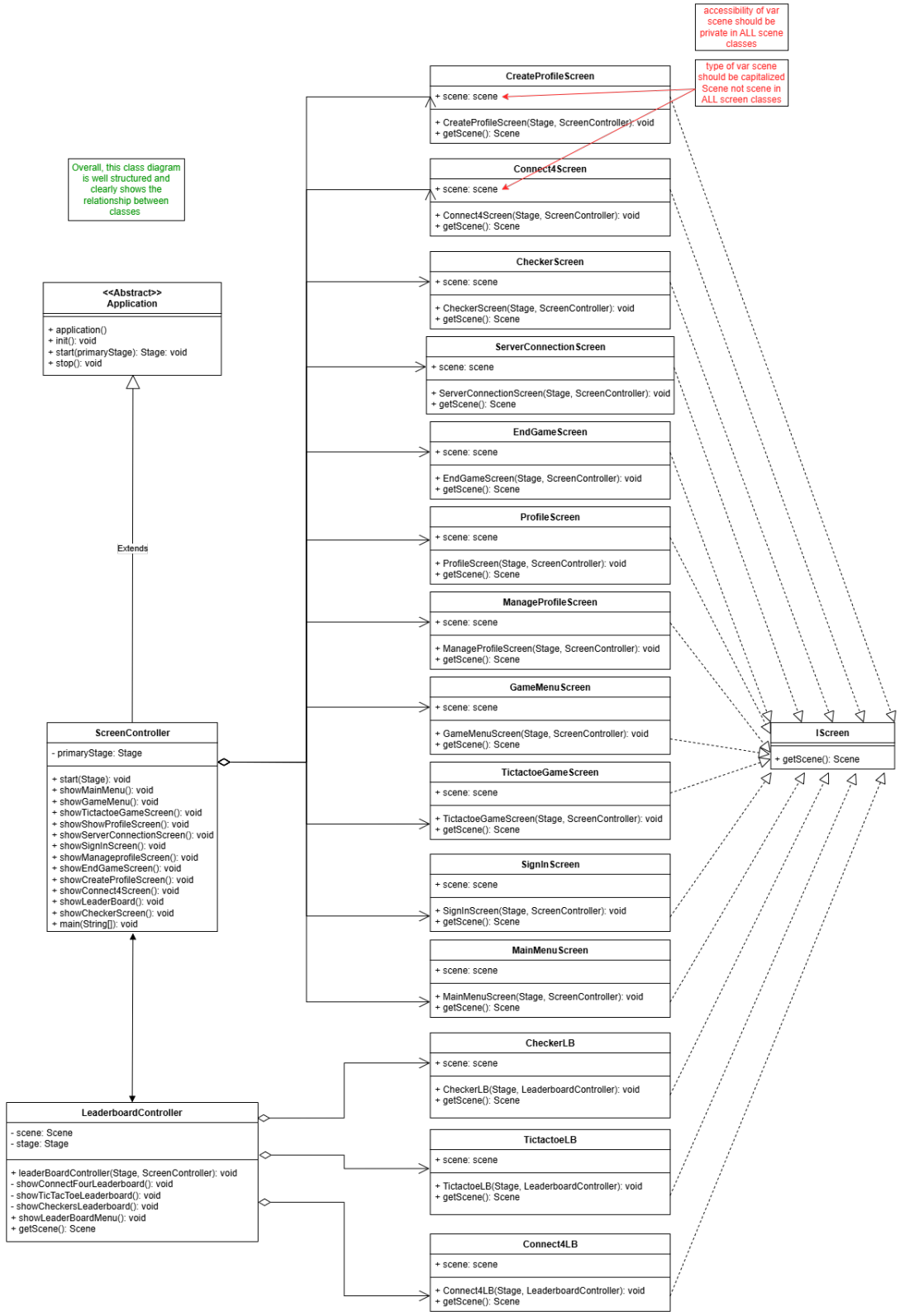
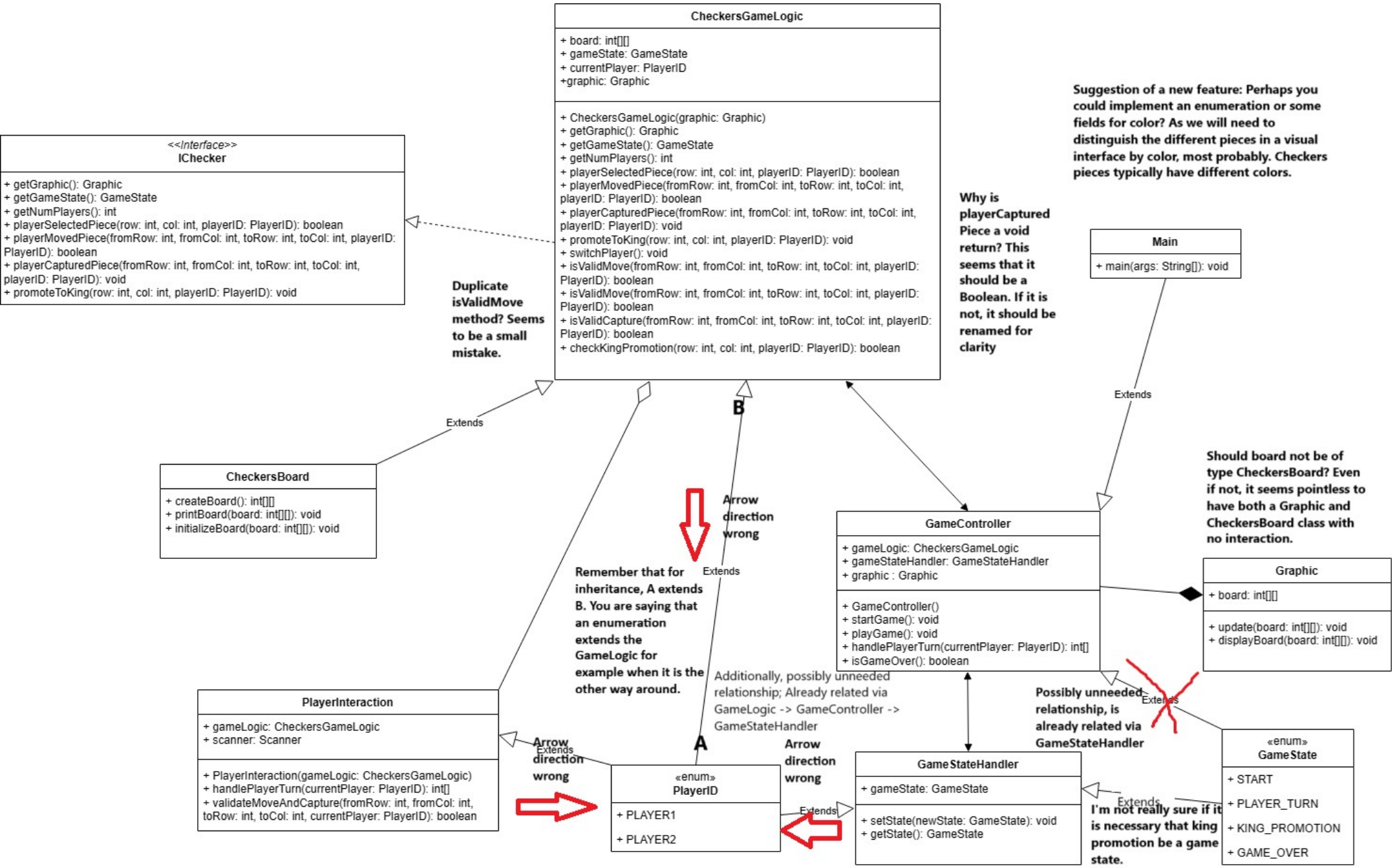
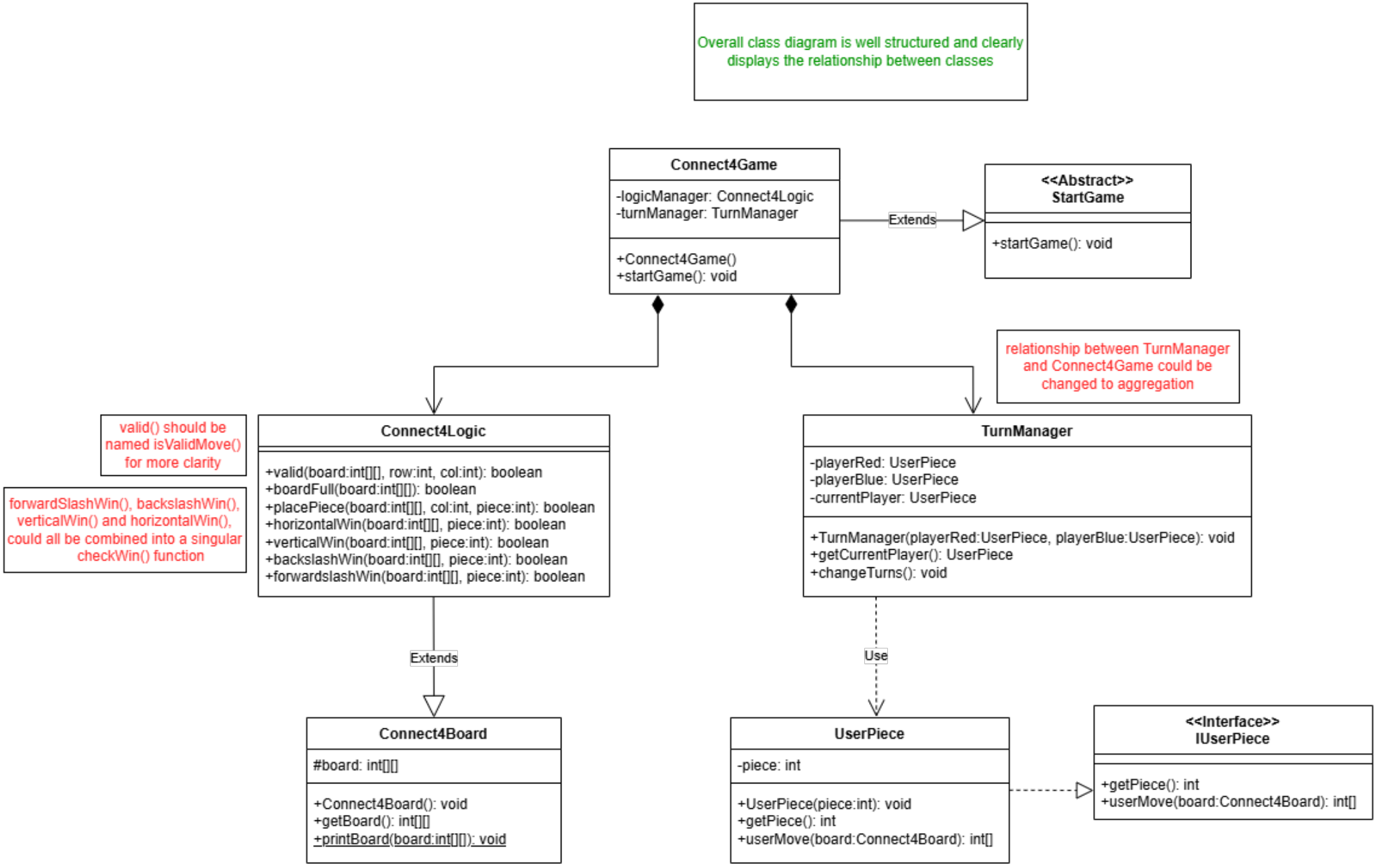
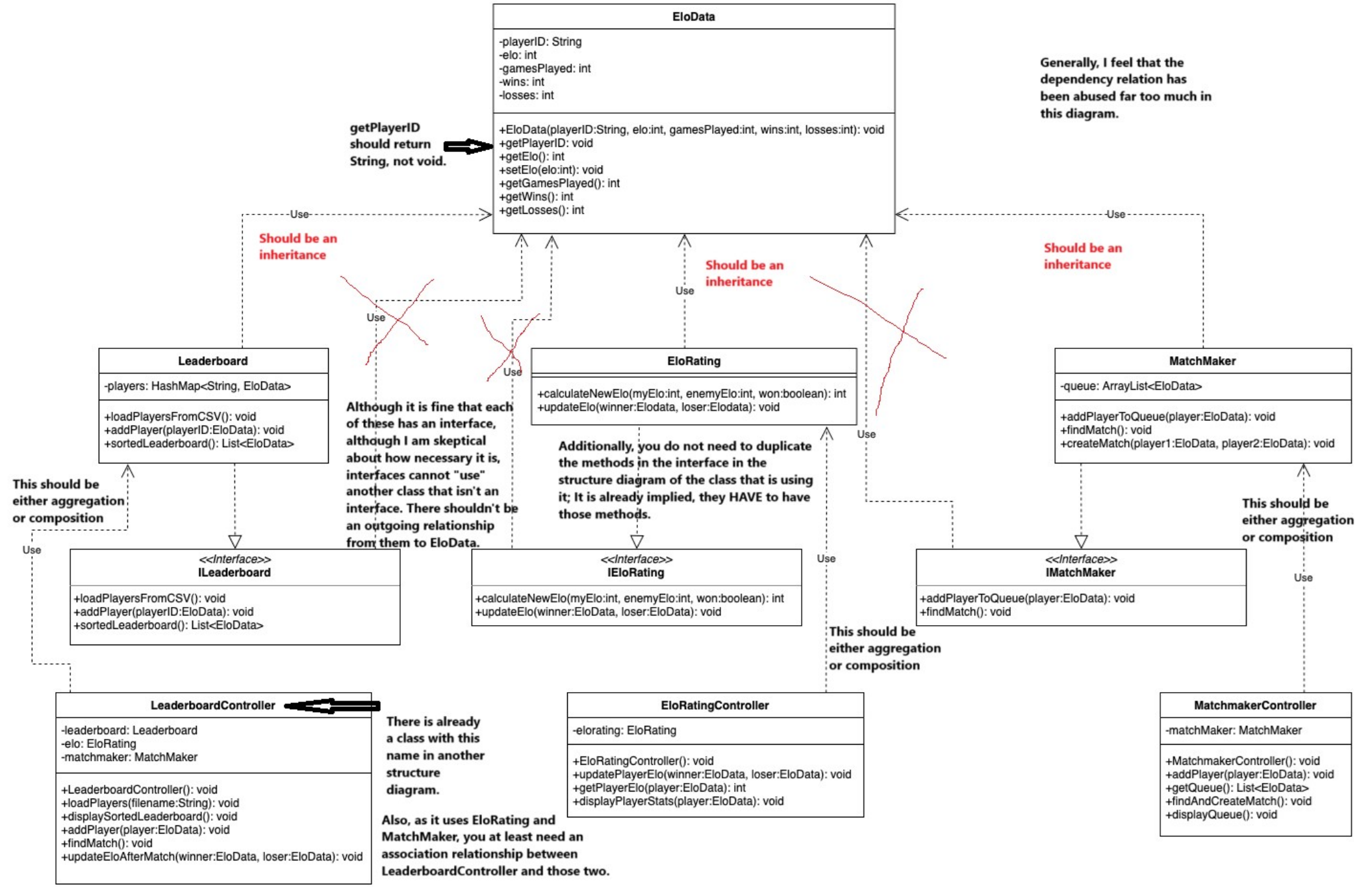
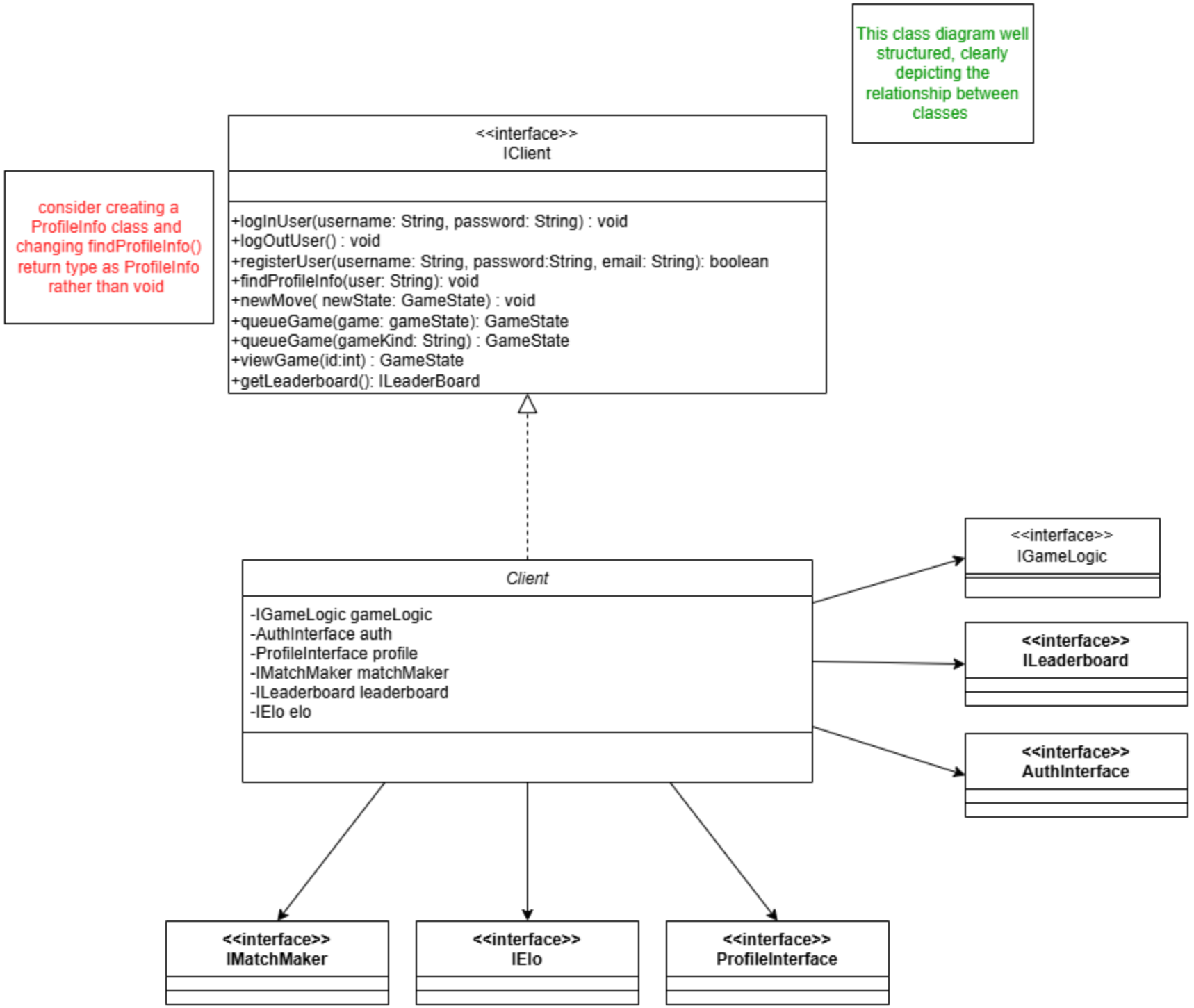

Bi-Directional Connector means communication both ways
Bi-Directional dotted Connector means functionality relationship
Diagram Description/Overview
This is a Project Infrastructure Diagram that will briefly explain how each required functionality will communicate with each other.
Suggestion: A more detailed GUI section could be added to match the complexity of Network. Such as "Main Menu", "chat" or "gameview"
GUI
GUI will communicate with Game logic to get game functionality
Suggestion: More details could be added to the game logic section to match the complexity of network
TicTacToe
Player interacts with GUI, it will be communicated to Network, and return information to GUI where Necessary
Game Logic
Connect4
Checkers
Feedback: Overall the general layout is very strong. Network is laid out comprehensively and is very detailed. Although the game logic and GUI sections are a bit lacking, but since this is just a general overview those suggestions are just up to preference. In addition more functional relationships can be added as outlined. Only including 2 relationships also makes the diagram a lot more readable. Also the relationship descriptions vastly help to clarify the diagram.
Suggestion: Adding a short descriptions to every relationship could be useful
Chat updated from Network and communicates back to GUI
any updates to board will be communicated to network to update GUI
Suggestion: Chat should have a functionality relationship with profile for fetching player info
Chat
Network
Suggestion: Leaderboard should have a functionality relationship with profile for fetching player info
Network will communicate with their respective databases and functions
Any profile communication will be queried from authenticaion database
Suggestion: Matchmaking should have a functional relationship with profile in order to queue players. Replacing the connection between "challenge players" and "Profile"
leaderboard
Matchmaking
Can search for player and challenge them
Profile
Profile Will have relationship with Authentication
Authentication
Leaderboard Includes game specific leaderboards
Authetication includes the following specific functions and databases
TicTacToe Leaderboard
TicTacToe Leaderboard
TicTacToe Leaderboard
Queue Game
Challenge Players
Profile Data
Log out
Log In
Register
Profile will include these types of databases
Error: Update each leaderboard name to match their respective games
Name
Match History
Stats
ELO

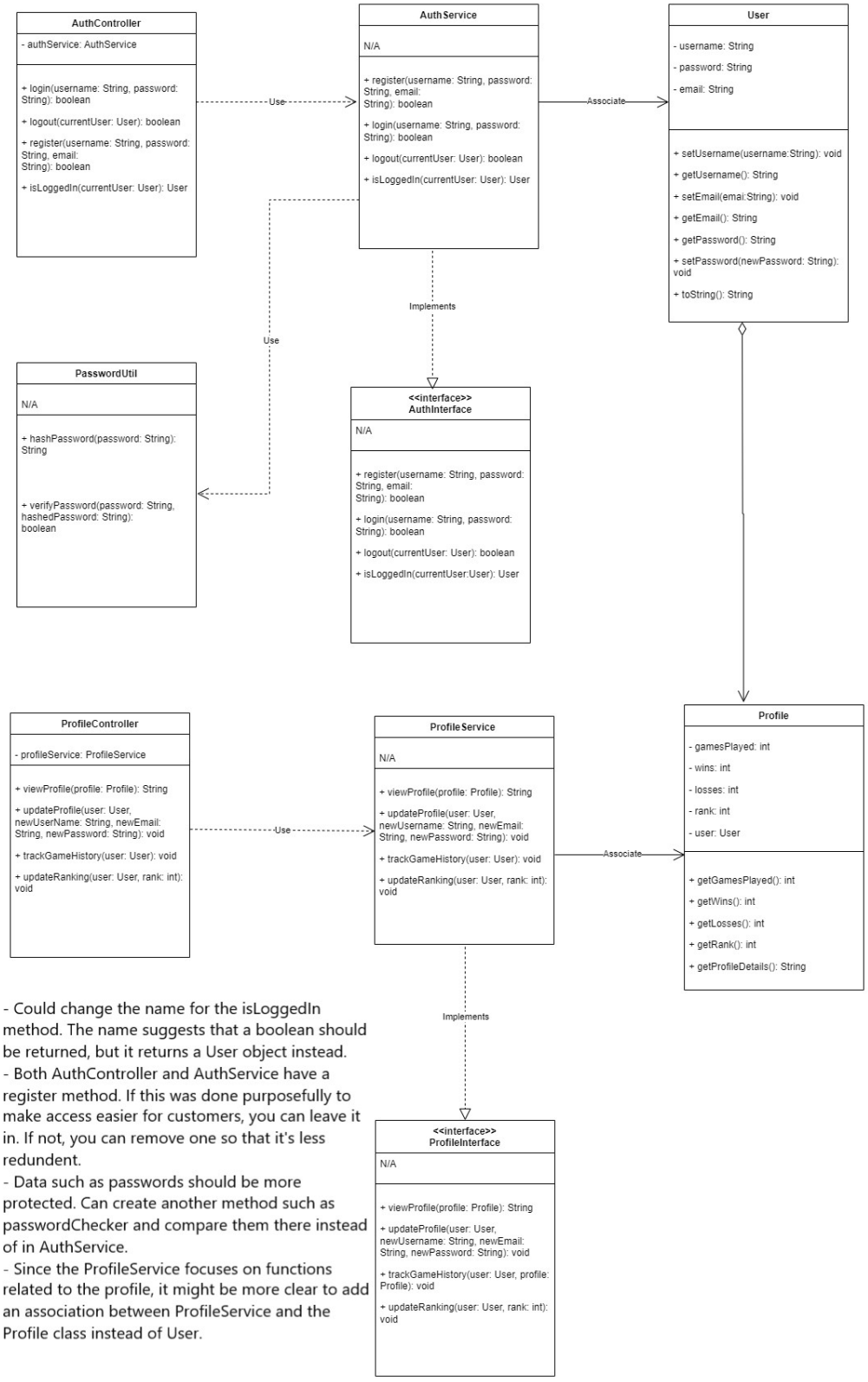

accessibility of var scene should be private in ALL scene classes
type of var scene should be capitalized Scene not scene in ALL screen classes
Overall, this class diagram is well structured and clearly shows the relationship between classes
CreateProfileScreen
+ scene: scene
+ CreateProfileScreen(Stage, ScreenController): void
+ getScene(): Scene
Connect4Screen
+ scene: scene
+ Connect4Screen(Stage, ScreenController): void
+ getScene(): Scene
CheckerScreen
+ scene: scene
+ CheckerScreen(Stage, ScreenController): void
+ getScene(): Scene
ServerConnectionScreen
+ scene: scene
+ ServerConnectionScreen(Stage, ScreenController): void
+ getScene(): Scene
EndGameScreen
+ scene: scene
+ EndGameScreen(Stage, ScreenController): void
+ getScene(): Scene
ProfileScreen
+ scene: scene
+ ProfileScreen(Stage, ScreenController): void
+ getScene(): Scene
ManageProfileScreen
+ scene: scene
+ ManageProfileScreen(Stage, ScreenController): void
+ getScene(): Scene
GameMenuScreen
+ scene: scene
+ GameMenuScreen(Stage, ScreenController): void
+ getScene(): Scene
TictactoeGameScreen
+ scene: scene
+ TictactoeGameScreen(Stage, ScreenController): void
+ getScene(): Scene
SignInScreen
+ scene: scene
+ SignInScreen(Stage, ScreenController): void
+ getScene(): Scene
MainMenuScreen
+ scene: scene
+ MainMenuScreen(Stage, ScreenController): void
+ getScene(): Scene
<<Abstract>>
Application
+ application()
+ init(): void
+ start(primaryStage): Stage: void
+ stop(): void
Extends
ScreenController
- primaryStage: Stage
+ start(Stage): void
+ showMainMenu(): void
+ showGameMenu(): void
+ showTictactoeGameScreen(): void
+ showShowProfileScreen(): void
+ showServerConnectionScreen(): void
+ showSignInScreen(): void
+ showManageprofileScreen(): void
+ showEndGameScreen(): void
+ showCreateProfileScreen(): void
+ showConnect4Screen(): void
+ showLeaderBoard(): void
+ showCheckerScreen(): void
+ main(String[]): void
IScreen
+ getScene(): Scene
LeaderboardController
- scene: Scene
- stage: Stage
+ leaderBoardController(Stage, ScreenController): void
- showConnectFourLeaderboard(): void
- showTicTacToeLeaderboard(): void
- showCheckersLeaderboard(): void
+ showLeaderBoardMenu(): void
+ getScene(): Scene
CheckerLB
+ scene: scene
+ CheckerLB(Stage, LeaderboardController): void
+ getScene(): Scene
TictactoeLB
+ scene: scene
+ TictactoeLB(Stage, LeaderboardController): void
+ getScene(): Scene
Connect4LB
+ scene: scene
+ Connect4LB(Stage, LeaderboardController): void
+ getScene(): Scene


CheckersGameLogic
+ board: int[][]
+ gameState: GameState
+ currentPlayer: PlayerID
+graphic: Graphic
+ CheckersGameLogic(graphic: Graphic)
+ getGraphic(): Graphic
+ getGameState(): GameState
+ getNumPlayers(): int
+ playerSelectedPiece(row: int, col: int, playerID: PlayerID): boolean
+ playerMovedPiece(fromRow: int, fromCol: int, toRow: int, toCol: int, playerID: PlayerID): boolean
+ playerCapturedPiece(fromRow: int, fromCol: int, toRow: int, toCol: int, playerID: PlayerID): void
+ promoteToKing(row: int, col: int, playerID: PlayerID): void
+ switchPlayer(): void
+ isValidMove(fromRow: int, fromCol: int, toRow: int, toCol: int, playerID: PlayerID): boolean
+ isValidMove(fromRow: int, fromCol: int, toRow: int, toCol: int, playerID: PlayerID): boolean
+ isValidCapture(fromRow: int, fromCol: int, toRow: int, toCol: int, playerID: PlayerID): boolean
+ checkKingPromotion(row: int, col: int, playerID: PlayerID): boolean
<<Interface>>
IChecker
+ getGraphic(): Graphic
+ getGameState(): GameState
+ getNumPlayers(): int
+ playerSelectedPiece(row: int, col: int, playerID: PlayerID): boolean
+ playerMovedPiece(fromRow: int, fromCol: int, toRow: int, toCol: int, playerID: PlayerID): boolean
+ playerCapturedPiece(fromRow: int, fromCol: int, toRow: int, toCol: int, playerID: PlayerID): void
+ promoteToKing(row: int, col: int, playerID: PlayerID): void
Duplicate isValidMove method? Seems to be a small mistake.
Why is playerCaptured Piece a void return? This seems that it should be a Boolean. If it is not, it should be renamed for clarity
Suggestion of a new feature: Perhaps you could implement an enumeration or some fields for color? As we will need to distinguish the different pieces in a visual interface by color, most probably. Checkers pieces typically have different colors.
Main
+ main(args: String[]): void
Extends
CheckersBoard
+ createBoard(): int[][]
+ printBoard(board: int[][]): void
+ initializeBoard(board: int[][]): void
B
Arrow direction wrong
Remember that for inheritance, A extends B. You are saying that an enumeration extends the GameLogic for example when it is the other way around.
Extends
GameController
+ gameLogic: CheckersGameLogic
+ gameStateHandler: GameStateHandler
+ graphic : Graphic
+ GameController()
+ startGame(): void
+ playGame(): void
+ handlePlayerTurn(currentPlayer: PlayerID): int[]
+ isGameOver(): boolean
Should board not be of type CheckersBoard? Even if not, it seems pointless to have both a Graphic and CheckersBoard class with no interaction.
Graphic
+ board: int[][]
+ update(board: int[][]): void
+ displayBoard(board: int[][]): void
Additionally, possibly unneeded relationship; Already related via GameLogic -> GameController -> GameStateHandler
Possibly unneeded relationship, is already related via GameStateHandler
Extends
PlayerInteraction
+ gameLogic: CheckersGameLogic
+ scanner: Scanner
+ PlayerInteraction(gameLogic: CheckersGameLogic)
+ handlePlayerTurn(currentPlayer: PlayerID): int[]
+ validateMoveAndCapture(fromRow: int, fromCol: int, toRow: int, toCol: int, currentPlayer: PlayerID): boolean
Arrow direction wrong
Extends
A
«enum»
PlayerID
+ PLAYER1
+ PLAYER2
Arrow direction wrong
Extends
GameStateHandler
+ gameState: GameState
+ setState(newState: GameState): void
+ getState(): GameState
Extends
«enum»
GameState
+ START
+ PLAYER_TURN
+ KING_PROMOTION
+ GAME_OVER
I'm not really sure if it is necessary that king promotion be a game state.


Overall class diagram is well structured and clearly displays the relationship between classes
Connect4Game
-logicManager: Connect4Logic
-turnManager: TurnManager
+Connect4Game()
+startGame(): void
Extends
<<Abstract>>
StartGame
+startGame(): void
relationship between TurnManager and Connect4Game could be changed to aggregation
valid() should be named isValidMove() for more clarity
forwardSlashWin(), backslashWin(), verticalWin() and horizontalWin(), could all be combined into a singular checkWin() function
Connect4Logic
+valid(board:int[][], row:int, col:int): boolean
+boardFull(board:int[][]): boolean
+placePiece(board:int[][], col:int, piece:int): boolean
+horizontalWin(board:int[][], piece:int): boolean
+verticalWin(board:int[][], piece:int): boolean
+backslashWin(board:int[][], piece:int): boolean
+forwardslashWin(board:int[][], piece:int): boolean
TurnManager
-playerRed: UserPiece
-playerBlue: UserPiece
-currentPlayer: UserPiece
+TurnManager(playerRed:UserPiece, playerBlue:UserPiece): void
+getCurrentPlayer(): UserPiece
+changeTurns(): void
Extends
Use
Connect4Board
#board: int[][]
+Connect4Board(): void
+getBoard(): int[][]
+printBoard(board:int[][]): void
UserPiece
-piece: int
+UserPiece(piece:int): void
+getPiece(): int
+userMove(board:Connect4Board): int[]
<<Interface>>
IUserPiece
+getPiece(): int
+userMove(board:Connect4Board): int[]


EloData
-playerID: String
-elo: int
-gamesPlayed: int
-wins: int
-losses: int
+EloData(playerID:String, elo:int, gamesPlayed:int, wins:int, losses:int): void
+getPlayerID: void
+getElo(): int
+setElo(elo:int): void
+getGamesPlayed(): int
+getWins(): int
+getLosses(): int
getPlayerID should return String, not void.
Generally, I feel that the dependency relation has been abused far too much in this diagram.
Use
Should be an inheritance
Leaderboard
-players: HashMap<String, EloData>
+loadPlayersFromCSV(): void
+addPlayer(playerID:EloData): void
+sortedLeaderboard(): List<EloData>
EloRating
+calculateNewElo(myElo:int, enemyElo:int, won:boolean): int
+updateElo(winner:Elodata, loser:Elodata): void
MatchMaker
-queue: ArrayList<EloData>
+addPlayerToQueue(player:EloData): void
+findMatch(): void
+createMatch(player1:EloData, player2:EloData): void
Although it is fine that each of these has an interface, although I am skeptical about how necessary it is, interfaces cannot "use" another class that isn't an interface. There shouldn't be an outgoing relationship from them to EloData.
Additionally, you do not need to duplicate the methods in the interface in the structure diagram of the class that is using it; It is already implied, they HAVE to have those methods.
This should be either aggregation or composition
<<Interface>>
ILeaderboard
+loadPlayersFromCSV(): void
+addPlayer(playerID:EloData): void
+sortedLeaderboard(): List<EloData>
<<Interface>>
IEloRating
+calculateNewElo(myElo:int, enemyElo:int, won:boolean): int
+updateElo(winner:EloData, loser:EloData): void
<<Interface>>
IMatchMaker
+addPlayerToQueue(player:EloData): void
+findMatch(): void
LeaderboardController
-leaderboard: Leaderboard
-elo: EloRating
-matchmaker: MatchMaker
+LeaderboardController(): void
+loadPlayers(filename:String): void
+displaySortedLeaderboard(): void
+addPlayer(player:EloData): void
+findMatch(): void
+updateEloAfterMatch(winner:EloData, loser:EloData): void
There is already a class with this name in another structure diagram.
Also, as it uses EloRating and MatchMaker, you at least need an association relationship between LeaderboardController and those two.
EloRatingController
-elorating: EloRating
+EloRatingController(): void
+updatePlayerElo(winner:EloData, loser:EloData): void
+getPlayerElo(player:EloData): int
+displayPlayerStats(player:EloData): void
MatchmakerController
-matchMaker: MatchMaker
+MatchmakerController(): void
+addPlayer(player:EloData): void
+getQueue(): List<EloData>
+findAndCreateMatch(): void
+displayQueue(): void

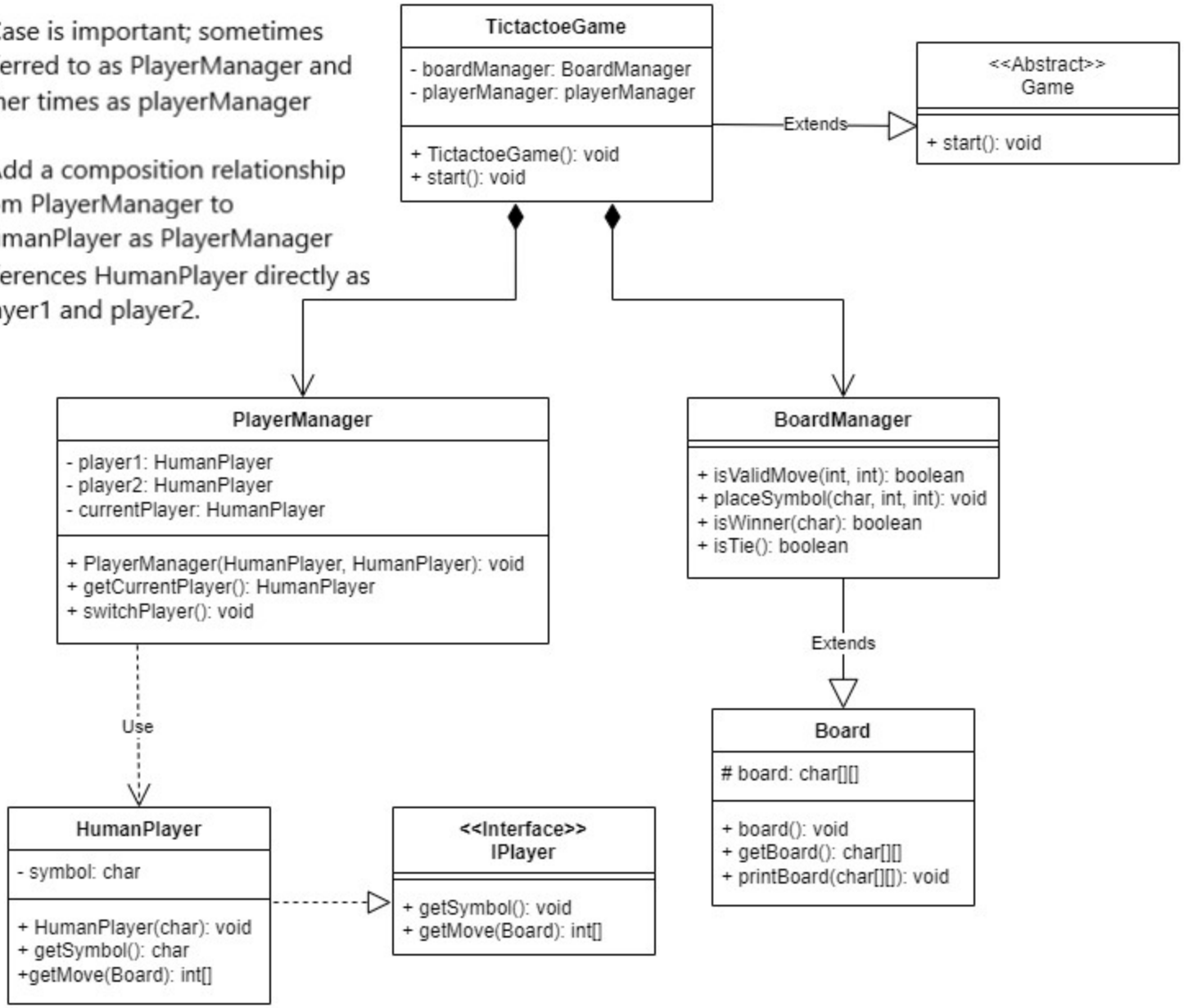

This class diagram well structured, clearly depicting the relationship between classes
consider creating a ProfileInfo class and changing findProfileInfo() return type as ProfileInfo rather than void
<<interface>>
IClient
+logInUser(username: String, password: String) : void
+logOutUser() : void
+registerUser(username: String, password:String, email: String): boolean
+findProfileInfo(user: String): void
+newMove(newState: GameState) : void
+queueGame(game: gameState): GameState
+queueGame(gameKind: String) : GameState
+viewGame(id:int) : GameState
+getLeaderboard(): ILeaderBoard
Client
-IGameLogic gameLogic
-AuthInterface auth
-ProfileInterface profile
-IMatchMaker matchMaker
-ILeaderboard leaderboard
-IElo elo
<<interface>>
IGameLogic
<<interface>>
ILeaderboard
<<interface>>
AuthInterface
<<interface>>
IMatchMaker
<<interface>>
IElo
<<interface>>
ProfileInterface

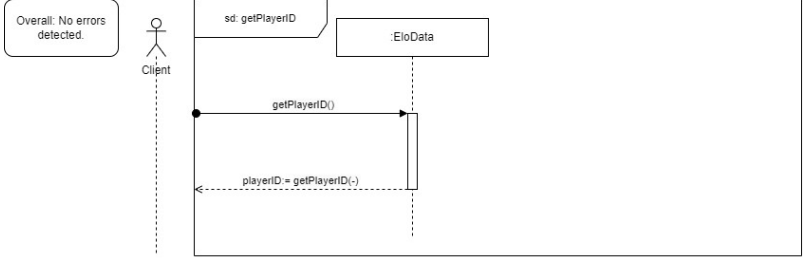


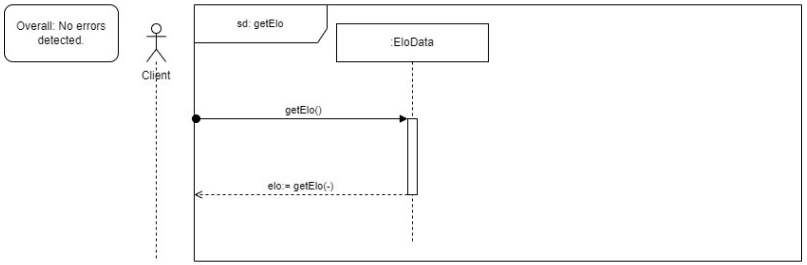
- Could change the name for the isLoggedIn method. The name suggests that a boolean should be returned, but it returns a User object instead.
- Both AuthController and AuthService have a register method. If this was done purposefully to make access easier for customers, you can leave it in. If not, you can remove one so that it's less redundent.
- Data such as passwords should be more protected. Can create another method such as passwordChecker and compare them there instead of in AuthService.
- Since the ProfileService focuses on functions related to the profile, it might be more clear to add an association between ProfileService and the Profile class instead of User.

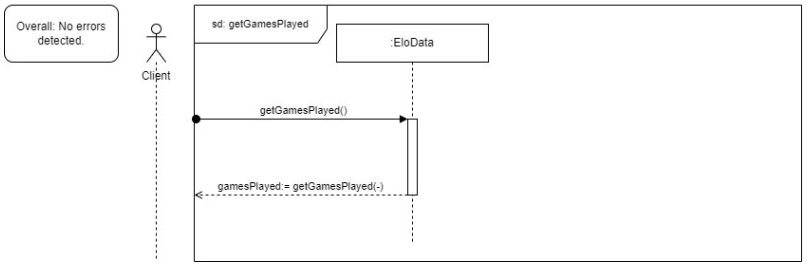
- Case is important; sometimes referred to as PlayerManager and other times as playerManager

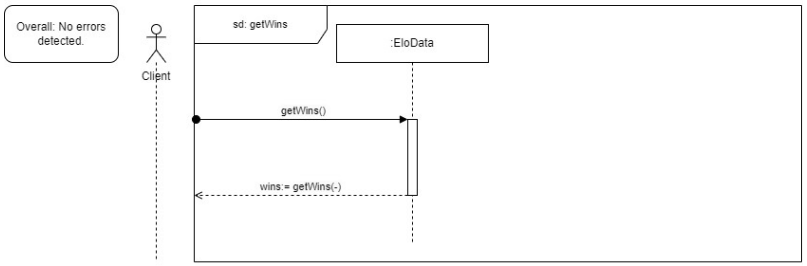
- Add a composition relationship from PlayerManager to HumanPlayer as PlayerManager references HumanPlayer directly as player1 and player2.

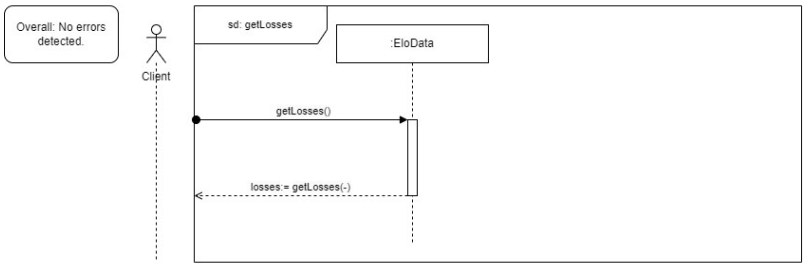


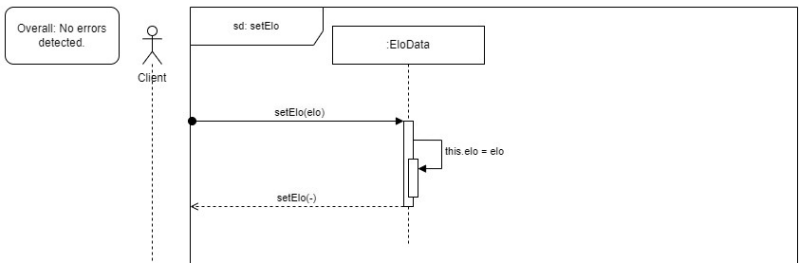

Overall: No errors detected.
Client
sd: getPlayerID
:EloData
getPlayerID()
playerID:= getPlayerID(-)

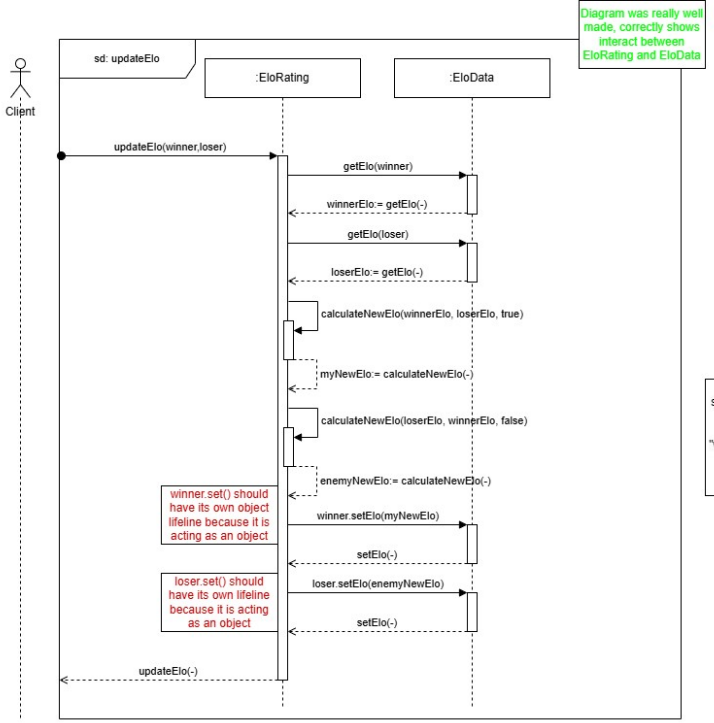

Overall: No errors detected.
Client
sd: getElo
:EloData
getElo()
elo:= getElo(-)

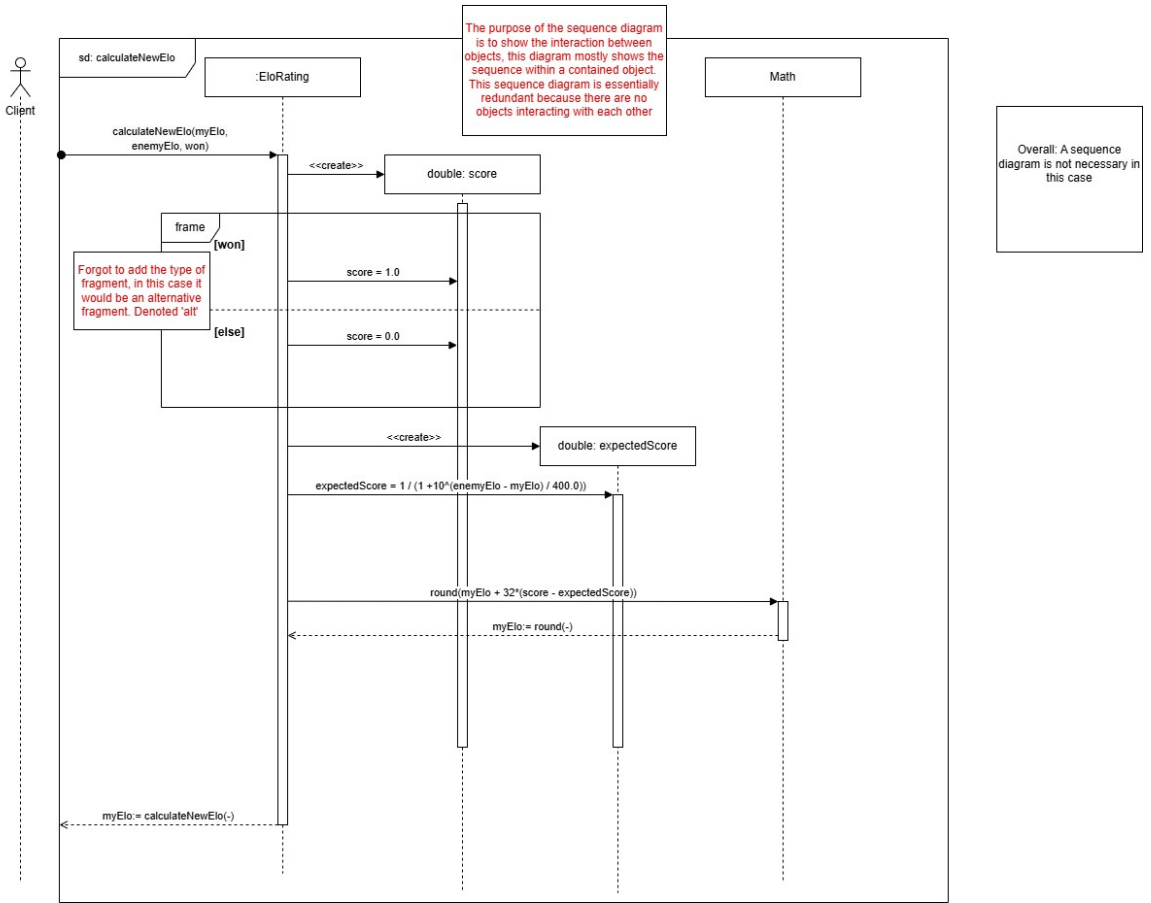

Overall: No errors detected.
Client
sd: getGamesPlayed
:EloData
getGamesPlayed()
gamesPlayed:= getGamesPlayed(-)

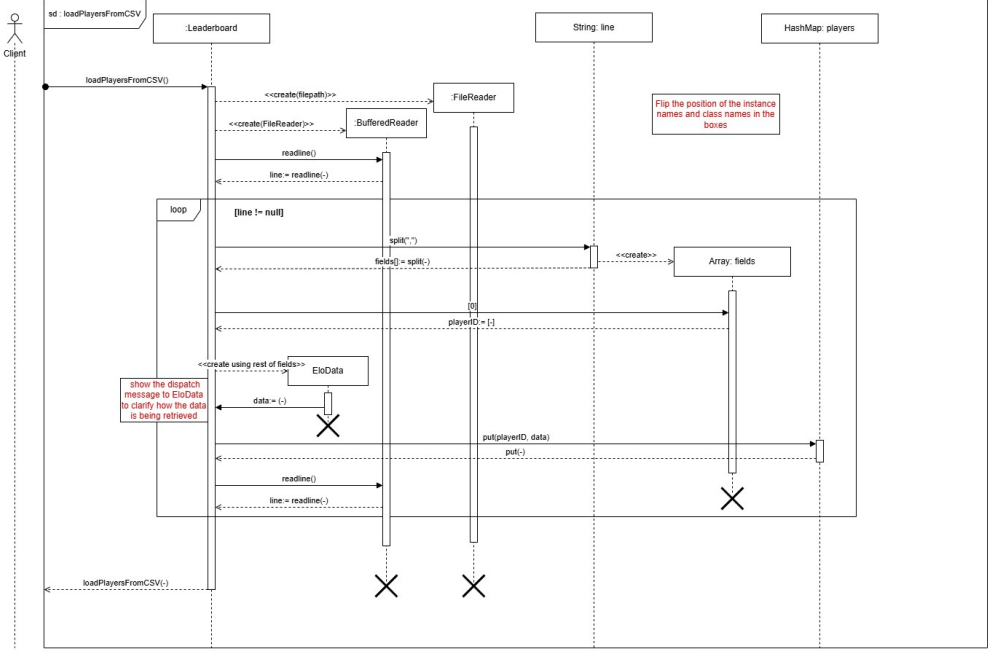

Overall: No errors detected.
Client
sd: getWins
:EloData
getWins()
wins:= getWins(-)


Overall: No errors detected.
Client
sd: getLosses
:EloData
getLosses()
losses:= getLosses(-)


Overall: No errors detected.
Client
sd: setElo
:EloData
setElo(elo)
this.elo = elo
setElo(-)

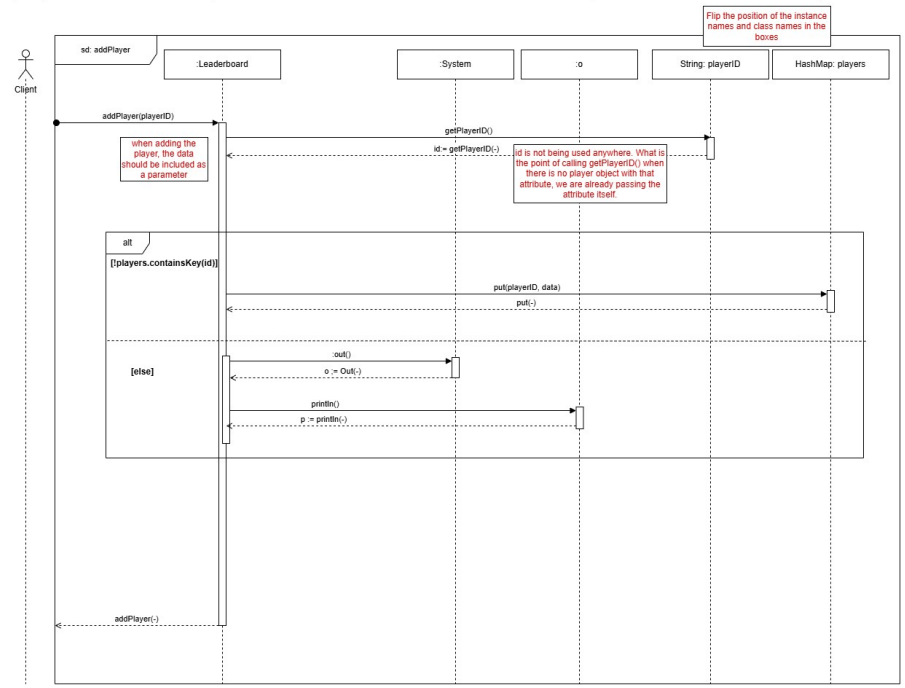

Diagram was really well made, correctly shows interact between EloRating and EloData
sd: updateElo
Client
:EloRating
:EloData
updateElo(winner,loser)
getElo(winner)
winnerElo:= getElo(-)
getElo(loser)
loserElo:= getElo(-)
calculateNewElo(winnerElo, loserElo, true)
myNewElo:= calculateNewElo(-)
calculateNewElo(loserElo, winnerElo, false)
enemyNewElo:= calculateNewElo(-)
winner.set() should have its own object lifeline because it is acting as an object
winner.setElo(myNewElo)
setElo(-)
loser.set() should have its own lifeline because it is acting as an object
loser.setElo(enemyNewElo)
setElo(-)
updateElo(-)
Overall: diagram is useful to the developers on the team and should be used as a blueprint.
suggestion: Instead of only having "EloData" itself. It can be modeled as if its branching off of player objects for example: "loser:Player" and "winner:Player" or however the player's attributes are stored lead to ":EloData".


The purpose of the sequence diagram is to show the interaction between objects, this diagram mostly shows the sequence within a contained object. This sequence diagram is essentially redundant because there are no objects interacting with each other
sd: calculateNewElo
Client
:EloRating
Math
calculateNewElo(myElo, enemyElo, won)
<<create>>
double: score
frame
[won]
Forgot to add the type of fragment, in this case it would be an alternative fragment. Denoted 'alt'
score = 1.0
[else]
score = 0.0
<<create>>
double: expectedScore
expectedScore = 1 / (1 +10^(enemyElo - myElo) / 400.0))
round(myElo + 32*(score - expectedScore))
myElo:= round(-)
myElo:= calculateNewElo(-)
Overall: A sequence diagram is not necessary in this case



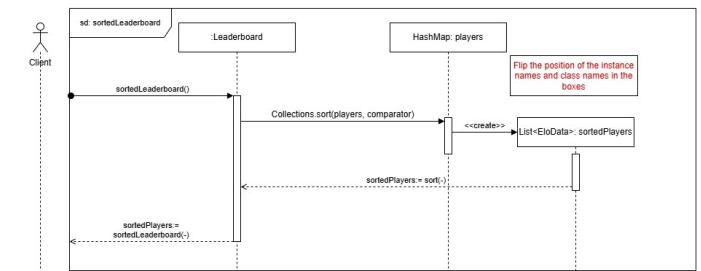
Overall: well done and shows interaction between multiple objects, clear sequence of events. Developers will have an easy time implementing this sequence

Feedback/suggestions: Ensure the object/role names are in the standard UML format . "Instance name : Class Name, clarify how the data is being retrieved



Overall: well done and shows interaction between multiple objects

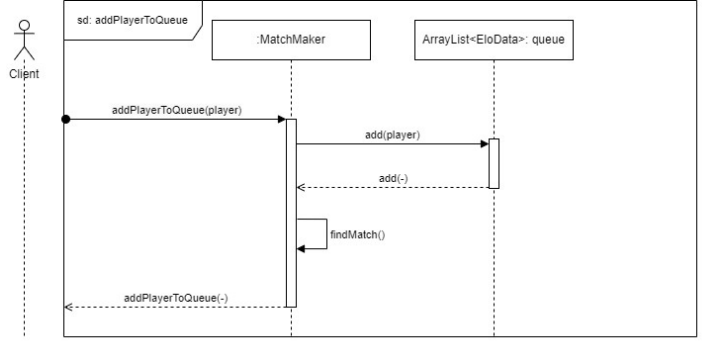
Feedback/suggestions: Ensure the object/role names are in the standard UML format . "Instance name : Class Name, review the getPlayerID() call, consider adding the player data as a parameter of addPlayer()



Overall: Good demonstration of how the Leaderboard objects interact with other objects in the sequence

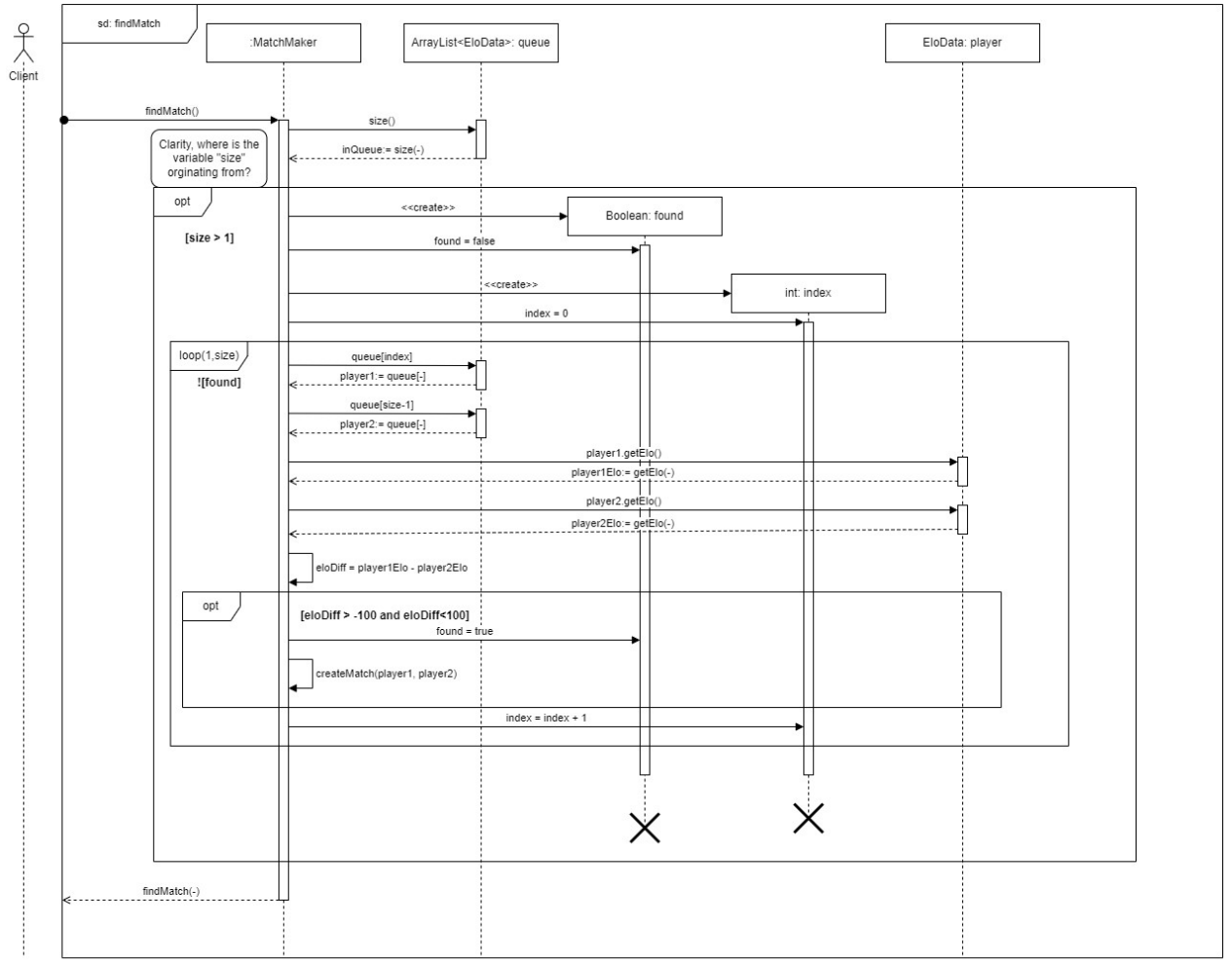
feedback: Ensure the object/role names are in the standard UML format . "Instance name : Class Name"

Overall: No diagram errors, just suggestions to possible enhance user experience.



Suggestion: Add a message indicating a player has been added to the queue, and a match is trying to be found? For example "You have been added to the Queue for [GAME]!"

Overall: No diagram errors, but there could be improvements in the code based on the diagram.



Optimization: player1 could be defined before entering the loop, rather than being re-defined for each iteration. This can drecrease resource consumption in the case of high player count in the queue.

Suggestion: Expand the boundary by a set amount if no player is within 100 elo. For example, after no elo of 100 is calculated, increase the boundary by 50 elo to find a player within 150 elo (keep growing until a player is found).

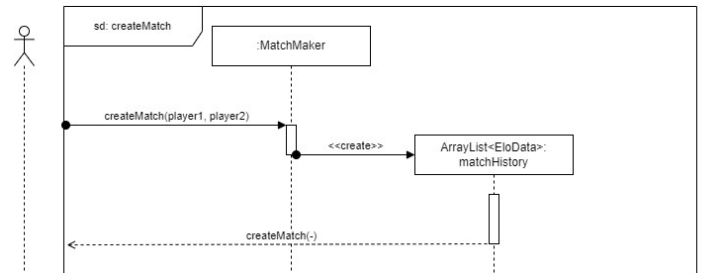
It would be inefficient if two players were only in the queue, but since their elo is not within 100 of each other, they are not paired, which creates player frustration and can decrease player count as players may be paired less frequently. However, there must be a mechanism that handles elo calculation due to widening elo differences, which suggests different skill levels between two players.

This can be implemented by having a set constant that increases the bound each time. To decrease runtime, have an array of all the calculated elo of the players already in the queue relative to the player.

Optimization: Instead of considering the negative and positive of the bound, use Math.abs(eloDiff). Do note you are importing an extra module, but using Math.abs(eloDiff) improves code readability.

Suggestion: If, for an unknown reason (such as game crashing, or not enough players), no match has been found, add a message that informs the player no match has been found. No message can easily confuse a game player waiting in queue.

Overall: No diagram errors.

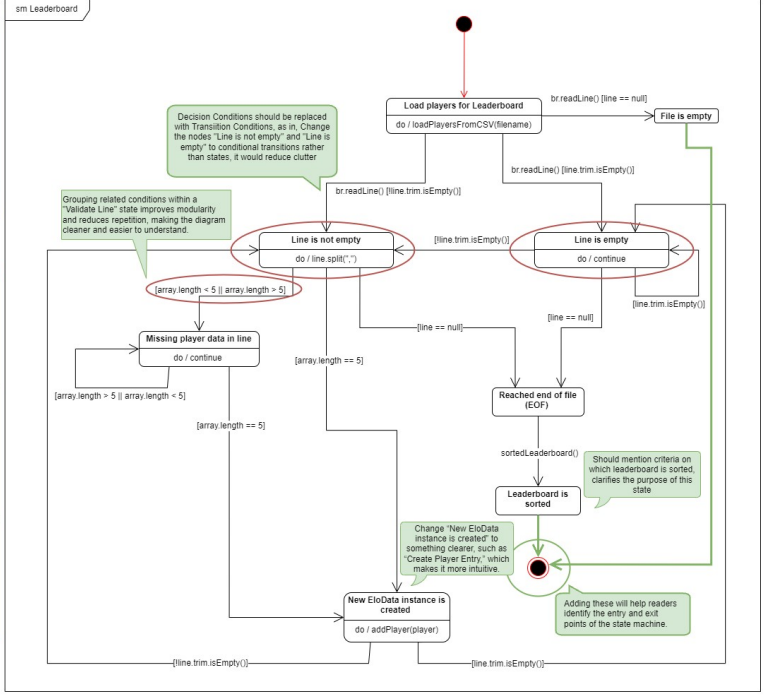


Strengths

1. Clear Steps for Data Loading: The state machine does a decent job at outlining the steps involved in handling data with certain conditions. This sequential approach is useful for understanding the basic file reading process
2. Basic Error Checking: The diagram includes basic validation checks, such as handlig empty lines and line lengths, which shows an effor to manage various file data conditions
3. Logic: Including sorting at the end of file reading process is a logical design choice, and the state machine diagram respects that.

Weaknesses

1. Cluttered and Redundant Conditions: There's a lot of redundancy in the conditions (e.g. line.trim.isEmpty(), array.length < 5 || array.length > 5). This repition not only clutters the diagram but also makes it harder to follow the logic. Streamlining these would improve clarity.
2. Lack of Clear terminators(Start and End): The absence of terminators makes the start and end of the process unclear, and some of the error handling states may lead to an infinite loop
3. Unclear terminology and ambiguous states: Some terminology, like "New EloData instance is created" and "Load Players for Leaderboard," could be made clearer to avoid confusion. Additionally, states like "Line is empty" and "Line is not empty" might be better represented as transitions rather than states.
4. Improper Use of State Machine Notation: Certain states like "Line is empty" and "Line is not empty" act more like decision points rather than distinct states. State machine diagrams typically focus on states and transitions between them, so using UML flowchart notation (with decision diamonds) might be more suitable in certain parts of the diagram.

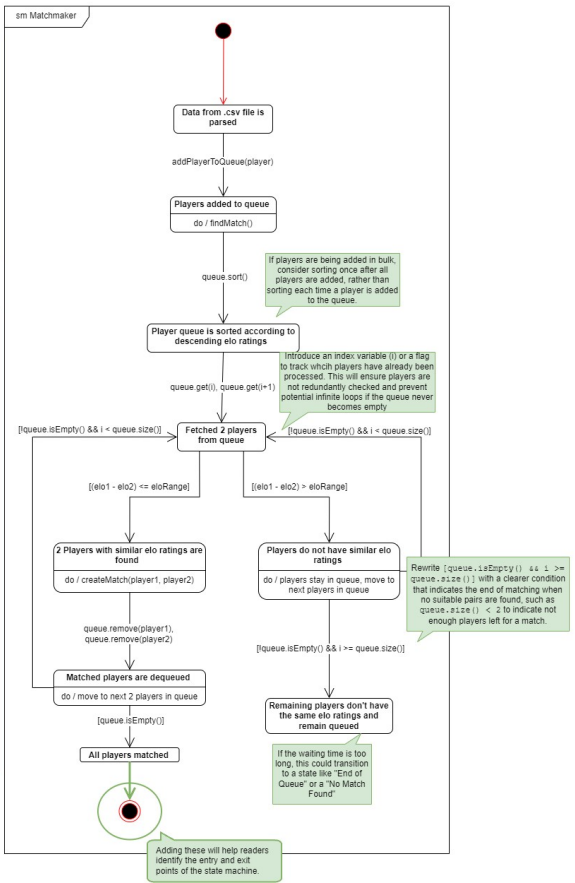


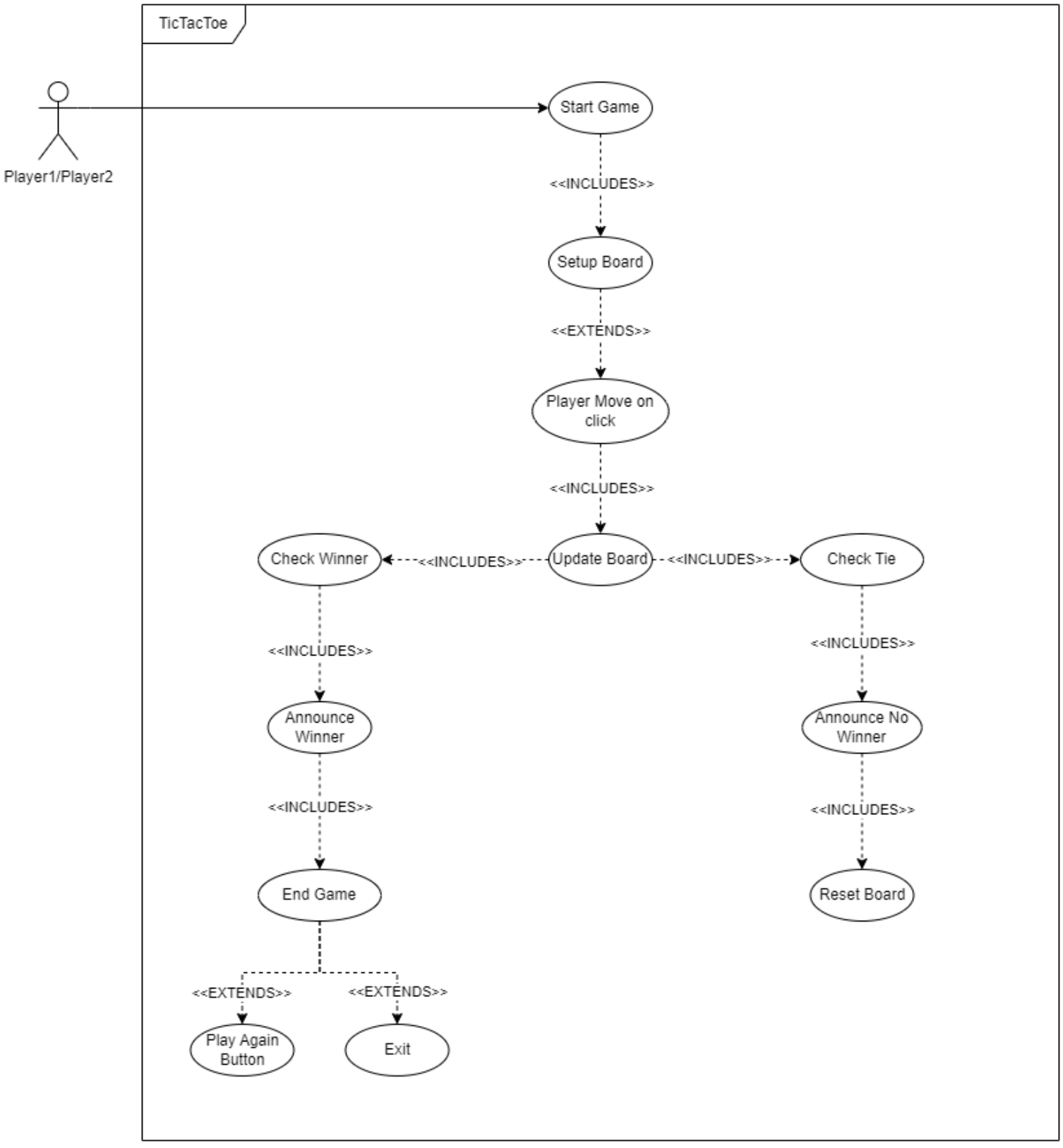
Strengths

1. Logical Structure: The diagram has a clear, linear flow, starting from parsing data and progressing through various stages of checking and matching players based on their Elo ratings.
2. Condition-based Branching: Conditions for checking the Elo rating difference (`[elo1 - elo2 <= eloRange]` and `[elo1 - elo2 > eloRange]`) are well-defined and help differentiate between similar and dissimilar rating pairings.
3. Terminology Clarity: The diagram uses understandable terminology such as "Players added to queue," "Player queue is sorted," and "Matched players are dequeued," which makes it relatively easy to follow the process.

Weaknesses

1. Missing terminators:The diagram lacks terminators at the start and end states. Adding an **End** terminator would clarify where the process definitively concludes, especially after "All players matched."
2. Loop Structure and Infinite Loop Risk: Similar to the previous diagram, this one could potentially lead to an infinite loop. If there are players left who do not meet the Elo range criteria, they stay in the queue indefinitely. Without a mechanism to exit or handle these cases, the process could continue endlessly.
3. Ambiguity in Queue Management: After players are "Fetched 2 players from queue," it's unclear if the state machine correctly tracks which pairs have already been checked. This could lead to repeatedly checking the same pairs if they don't meet the criteria, especially if the queue never becomes empty.
4. Inefficient Sorting Trigger: The state "Player queue is sorted according to descending Elo ratings" is reached each time after a player is added, which might be inefficient if triggered repeatedly for large numbers of players. It would be more efficient to sort the queue only once, after all players are added.
5. d.Confusing Condition Syntax: The condition `[queue.isEmpty() && i >= queue.size()]` in "Remaining players don't have the same Elo ratings and remain queued" could be misinterpreted. The meaning and purpose of `i` are unclear. Clarify whether `i` represents an index or a player count.




TicTacToe
Player1/Player2
Start Game
<<INCLUDES>>
Setup Board
<<EXTENDS>>
Player Move on click
<<INCLUDES>>
Update Board
<<INCLUDES>>
Check Winner
<<INCLUDES>>
Check Tie
<<INCLUDES>>
Announce Winner
<<INCLUDES>>
End Game
<<EXTENDS>>
Play Again Button
<<EXTENDS>>
Exit
<<INCLUDES>>
Announce No Winner
<<INCLUDES>>
Reset Board

Use Case Descriptions (Tic-Tac-Toe) 

Use Case: Start Game

Iteration: 1st iteration

Primary actor: Player1, Player2

Goal in context: Begins a new Tic-Tac-Toe game session

Preconditions: The system is powered on and user has selected Tic-Tac-Toe as their game of choice

Trigger: Either Player1 or Player2 (depending on host) selects Tic-Tac-Toe as their game of choice with the “Tic-Tac-Toe” button)

Scenario:

1. Player selects Tic-Tac-Toe game option
2. The system will initialize a Tic-Tac-Toe session
3. The session will create any required information before setting up the board (visualization of the game).

Post Conditions: Game is setup and ready to print board (visual) to users

Exceptions:

- The button for Tic-Tac-Toe was unresponsive
- The system encounters an error while initializing
    - Errors
    - Missing information

Priority: High - the first initialization of the game and setups are important to the functionality of GUI and game logic.
When Available: Within 1 sprint (1st iteration)
Frequency of Use: Once per game session
Channel to actor: Interaction of a click of the Tic-Tac-Toe game option with their mouse
Secondary actor: N/A.
Channel to Secondary Actors: N/A
Open issues:

- The setup of the game (any information required) is finished before GUI implementation

Use Case: Setup Board

Iteration: 1st iteration

Primary actor: Player1, Player2

Goal in context: The game was set up and the board is ready to be displayed to the user. The game will call a function to print the board.

Preconditions: The game was properly setup as required with the GUI ready for response from printBoard function.

Trigger: The system has set up the game and will now call a function for printing the board.

Scenario:

1. Game information is setup and properly initialized
2. Game calls to a printBoard function to display to user

Post Conditions: board is set up (the visual) and ready for Player1 to make their move

Exceptions:

- The function call was unresponsive
- The system encounters an error while starting the visual
    - Possibly incorrect formatting

Priority: High - visual representation should be set up in-order for users to interact with the game.
When Available: Within 1 sprint (1st iteration)
Frequency of Use: Once per game session
Channel to actor: When they have selected Tic-Tac-Toe as their game of choice (is part of initialization)
Secondary actor: N/A.
Channel to Secondary Actors: N/A
Open issues:

- Board was incorrectly setup

Use Case: Player Move on click

Iteration: 1st iteration

Primary actor: Player1, Player2

Goal in context: Player1/player2 can make their move by clicking any of the tiles of tic-tac-toe to place their symbol (X or O) onto the board.

Preconditions: The game board was set up and is ready for user input.

Trigger: The user has clicked a tile to place their piece.

Scenario:

1. The board is ready for user to input their piece
2. User clicks the tile they desire to place their piece
3. The piece is placed in the desired spot

Post Conditions: the user has successfully placed their piece onto the board and will now prompt the other user to place their piece

Exceptions:

- The piece was not placed in the correct spot
    - The piece not placed at all
- The system encounters an error placing piece

Priority: High - Placing their pieces is the main functionality of the Tic-Tac-Toe game.
When Available: Within 1 sprint (1st iteration)
Frequency of Use: N times per player
Channel to actor: Interaction of a click on the tile they desire to place their symbol with their mouse
Secondary actor: N/A
Channel to Secondary Actors: N/A
Open issues: N/A

Use Case: Update board

Iteration: 1st iteration

Primary actor: Player1, Player2

Goal in context: After any user input, the board is updated and displayed to the user.

Preconditions: users have selected their move and the board needs to be updated.

Trigger: user has selected their move.

Scenario:

1. Player has made their move
2. GUI collects this information and passes to game logic
3. New updated game board is passed back to GUI to display

Post Conditions: The new updated board is returned to GUI to display to users

Exceptions:

- Board was not updated properly
- An error has occurred
    - No updates to board

Priority: High - In order to understand what is happening in the game, it is important to continually update the board after each player input.
When Available: Within 1 sprint (1st iteration)
Frequency of Use: N times per player
Channel to actor: player has clicked a tile to make their move and system calls to update the board
Secondary actor: N/A
Channel to Secondary Actors: N/A
Open issues: N/A

Use Case: Check Winner

Iteration: 2nd iteration

Primary actor: Player1, Player2

Goal in context: After each player move, the system should check if there is a winner every turn

Preconditions: the user has placed a symbol on the board and the board was updated properly

Trigger: the board was updated.

Scenario:

1. The board was updated based on player's move
2. The system will check if a winner was detected

Post Conditions: system successfully checks if there was a winner detected

Exceptions:

- Board was updated incorrectly and checking winner returns incorrect information
- Conditions were never checked

Priority: High - determines the game conditions in order to win and end the game
When Available: Within 2 sprint (2st iteration)
Frequency of Use: N times per player
Channel to actor: once the board has been updated, system calls to check if there is a winner based on move
Secondary actor: N/A
Channel to Secondary Actors: N/A
Open issues:

- Game logic is incorrect

Use Case: Check Tie

Iteration: 2nd iteration

Primary actor: Player1, Player2

Goal in context: After each player moves the system should check if there is a tie after every turn

Preconditions: the user has placed a symbol on the board and the board was updated properly

Trigger: the board was updated

1. **Scenario:** The board was updated based on player's move
2. The system will check if a tie was detected

Post Conditions: system successfully checks if there was a tie detected

Exceptions:

- Board was updated incorrectly and checking tie returns incorrect information
- Conditions were never checked

Priority: high determines if a tie was detected and the game should end
When Available: Within 2 sprint (2st iteration)
Frequency of Use: N times per player
Channel to actor: once the board has been updated, system calls to check if there is a tie based on move
Secondary actor: N/A
Channel to Secondary Actors: N/A
Open issues:

- Game logic is incorrect

Use Case: Announce Winner

Iteration: 2nd iteration

Primary actor: Player1, Player2

Goal in context: A winner was found and a winner will be announced and displayed to GUI

Preconditions: a winner was found

Trigger: System has detected a winner

Scenario:

1. Winning conditions were found
2. A winner announcement is prompted to GUI

Post Conditions: A winner announcement is prompted to GUI

Exceptions:

- Announces incorrect prompt
    - Announces incorrect winner

Priority: Low - can be implemented at any stage of the program as long as winning conditions are correct
When Available: Within 2 sprint (2st iteration)
Frequency of Use: once per game
Channel to actor: winner was found and system announces winner to GUI
Secondary actor: N/A
Channel to Secondary Actors: N/A
Open issues: N/A

Use Case: Announce No winner

Iteration: 2nd iteration

Primary actor: Player1, Player2

Goal in context: No winner was found (board is full) and will be announced and displayed to GUI

Preconditions: no winner was found

Trigger: no winner was found

Scenario:

3. No winner was found
4. A no winner announcement is prompted to GUI

Post Conditions: A no winner announcement is prompted to GUI

Exceptions:

- Announces incorrect prompt

Priority: Low - can be implemented at any stage of the program as long as tie conditions are correct
When Available: Within 2 sprint (2st iteration)
Frequency of Use: once per game
Channel to actor: A tie game was found and system announces tie to GUI
Secondary actor: N/A
Channel to Secondary Actors: N/A
Open issues: N/A

Use Case: Reset Board

Iteration: 2nd iteration

Primary actor: Player1, Player2

Goal in context: create an empty board to reset the game

Preconditions: a previous game has ended

Trigger: “Restart” button

Scenario: 1. A previous game has ended
2.The user decides to start a new game

Post Conditions:
The game was properly setup as required with the GUI ready for response from printBoard function.

Trigger: The system has set up the game and will now call a function for printing the board.

Scenario:

3. Game information is setup and properly initialized
4. Game calls to a printBoard function to display to user

Post Conditions: board is set up (the visual) and ready for Player1 to make their move
Exceptions:

- The function call was unresponsive
- The system encounters an error while starting the visual
    - Possibly incorrect formatting

Priority: High - visual representation should be set up in-order for users to interact with the game.
When Available: Within 1 sprint (1st iteration)
Frequency of Use: Once per game session
Channel to actor: When they have selected Tic-Tac-Toe as their game of choice (is part of initialization)
Secondary actor: N/A.
Channel to Secondary Actors: N/A
Open issues:

- Board was incorrectly setup

Use Case: End Game

Iteration: 2nd iteration

Primary actor: Player 1, Play 

Goal in context: end game

Preconditions: The game board is initialized.

- The players have taken turns placing their symbols (X or O) on the board.

Trigger: a tie or win has occurred

Scenario: players placed their marks on cells and a win or tie was achieved and the game is over

Post Conditions: restart or exit 

Exceptions: If the board is in an invalid state (e.g., a player has already won but more moves are made), reset or display an error.

Priority: high- ends the game in finite time
When Available: 1st iteration
Frequency of Use: once each game
Channel to actor: on screen information about the previous game 
Secondary actor: N/A
Channel to Secondary Actors: N/A
Open issues: win or tie was incorrectly detected

Use Case: Play Again Button

Iteration: 2nd iteration

Primary actor: Player1, Player2

Goal in context: reset board on button click

Preconditions: the game has ended

Trigger: user clicks “play again”

Scenario: the game ends the user is given options to “play again” or “exit”

Post Conditions: the board is re-initialized

Exceptions: the game ended unexpectedly

Priority: high- allows for continued play
When Available: 1st iteration
Frequency of Use: up to once per game
Channel to actor: button on screen when game ends
Secondary actor: N/A
Channel to Secondary Actors: N/A
Open issues: game ends unexpectedly because a win or tie was detected incorrectly

Use Case: Exit

Iteration: 2nd iteration

Primary actor: Player1, Player2

Goal in context: exit game on button click

Preconditions: the game has ended

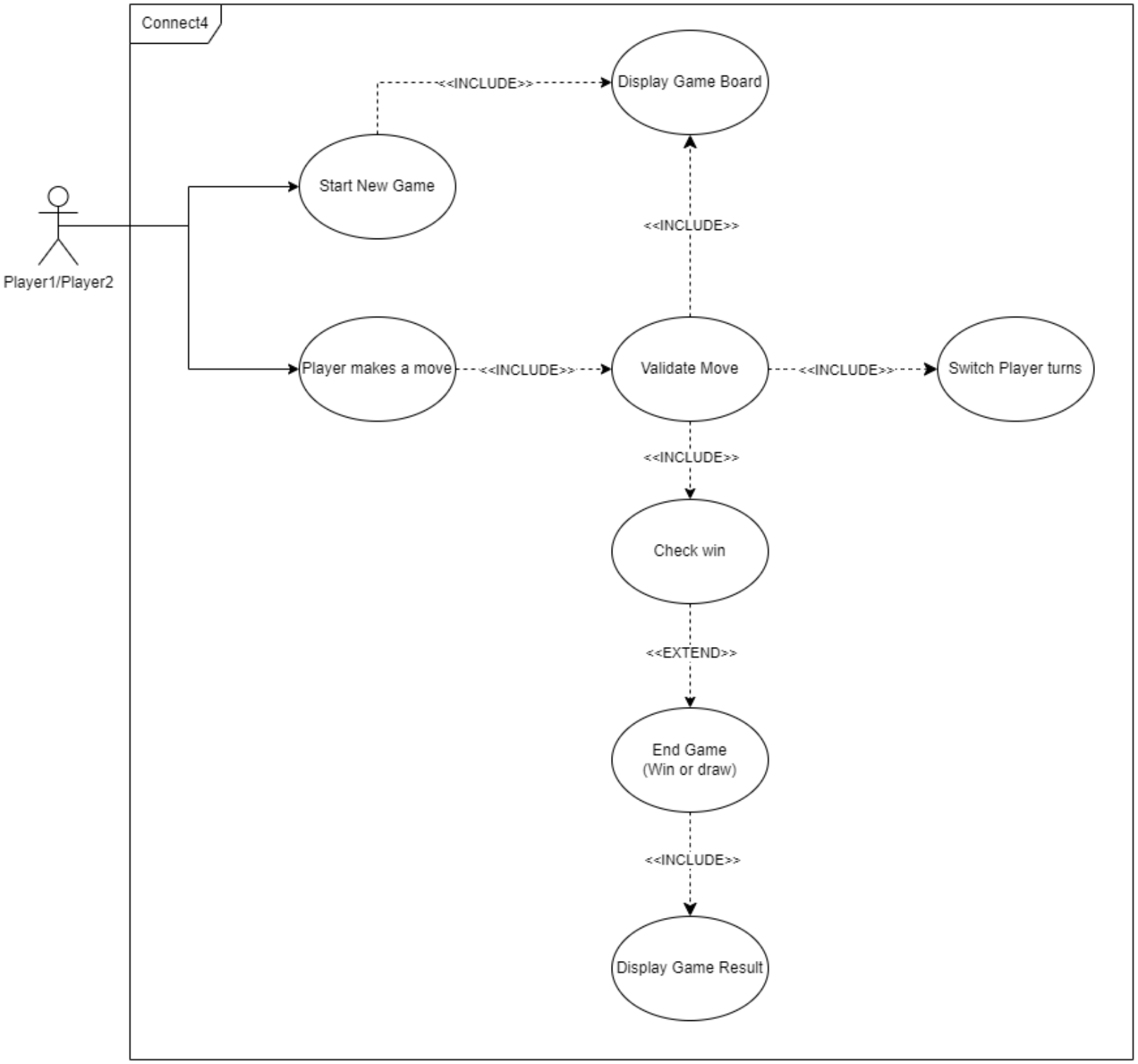
Trigger: user clicks “Exit”

Scenario: the game ends the user is given options to “play again” or “exit”

Post Conditions: the game is exited the user is given options to play a different game such as connect4 or checkers

Exceptions: the game ended unexpectedly

Priority: high- allows for continued play
When Available: 1st iteration
Frequency of Use: up to once per game
Channel to actor: button on screen when game ends
Secondary actor: N/A
Channel to Secondary Actors: N/A
Open issues: game ends unexpectedly because a win or tie was detected incorrectly


Connect4
Display Game Board
<<INCLUDE>>
Start New Game
Player1/Player2
<<INCLUDE>>
Player makes a move
<<INCLUDE>>
Validate Move
<<INCLUDE>>
Switch Player turns
<<INCLUDE>>
Check win
<<EXTEND>>
End Game
(Win or draw)
<<INCLUDE>>
Display Game Result

Use Case Descriptions (Connect4)

Use Case: Start New Game

Iteration: 1

Primary Actor: Player

Goal in context: To initiate a new game, resetting the game board and allowing players to begin a fresh round.

Preconditions: The application is active and ready for a new game.

Trigger: The player selects the option to start a new game.

Scenario:

1. The player chooses to start a new game.
2. The system clears the game board and prepares it for new moves.
3. The initial state of the game board is displayed.

Postconditions: The game board is reset and ready for the first move.

Exceptions: None.

Priority: High – Essential for game functionality.

When available: Within 1 sprint (first iteration).

Frequency of use: At the beginning of each new game.

Channel to actor: Visual confirmation of a new game board.

Secondary actors: N/A

Channel to secondary actors: N/A

Open issues: N/A

Use Case: Display Game Board

Iteration: 1

Primary Actor: Player

Goal in context: To provide a visual display of the current game board, enabling players to understand the game state.

Preconditions: The game has started, and at least one move has been made.

Trigger: The game begins or a move is completed.

Scenario:

1. The system generates and displays a visual representation of the board.
2. Column numbers and current piece placements are shown.
3. The system updates the display after each move.

Postconditions: The board is continuously visible, reflecting the latest moves.

Exceptions: Display errors that prevent the board from being shown accurately.

Priority: High – Vital for game clarity.

When available: Within 1 sprint (first iteration).

Frequency of use: Continuously, after each move and at game start.

Channel to actor: Visual display of the board.

Secondary actors: N/A

Channel to secondary actors: N/A

Open issues: N/A

Use Case: Player Makes a Move

Iteration: 1

Primary Actor: Player

Goal in context: To drop a piece in a chosen column, advancing the game.

Preconditions: The game is in progress, and it is the player's turn.

Trigger: The player selects a column for their move.

Scenario:

1. The player selects a column to place their piece.
2. The system validates the move for legality.
3. If valid, the piece is added to the board in the selected column.

Postconditions: The move is recorded, and the board reflects the new state.

Exceptions: If the column is full, prompt the player to select another column.

Priority: High – Core game mechanic.

When available: Within 1 sprint (first iteration).

Frequency of use: Repeatedly, throughout the game.

Channel to actor: Confirmation of the move. 

Secondary actors: N/A

Channel to secondary actors: N/A

Open issues: None

Use Case: Validate Move

Iteration: 1

Primary Actor: Player

Goal in context: To ensure the move chosen is valid and can be executed on the board.

Preconditions: The player has selected a column for their move.

Trigger: The player attempts a move.

Scenario:

1. The player's move triggers a check if the selected column has space for a new piece.
2. If the move is valid, the system allows the piece to be placed.
3. If not, the system prompts the player to select another column.

Postconditions: Only valid moves are executed, preventing errors.

Exceptions: Full columns prevent the move; the player is prompted to try again.

Priority: High – Ensures fair gameplay.

When available: Within 1 sprint (first iteration).

Frequency of use: Every time a player makes a move.

Channel to actor: Visual prompt if the move is invalid.

Secondary actors: N/A

Channel to secondary actors: N/A

Open issues: None.

Use Case: Check Win

Iteration: 1

Primary Actor: Player

Goal in context: To determine if a winning condition has been met after making a move.

Preconditions: A valid move has been made.

Trigger: Completion of the player's move.

Scenario:

1. The player's move triggers an analysis of the board for four consecutive pieces by the same player.
2. If a winning condition is met, it triggers the end game.
3. If no winning condition is met, the game continues.

Postconditions: The game state is updated based on the win check.

Exceptions: None. 

Priority: High – Critical for game outcome.

When available: Within 1 sprint (first iteration).

Frequency of use: After every move.

Channel to actor: N/A (system-initiated).

Secondary actors: N/A

Channel to secondary actors: N/A

Open issues: None.

Use Case: End Game (Win or Draw)

Iteration: 1

Primary Actor: Player

Goal in context: To conclude the game if a player has won or if the board is full, resulting in a draw.

Preconditions: A win condition is met, or the board is full.

Trigger: A win is detected or no moves are left.

Scenario:

1. The player's move or full board triggers the end game.
2. The system halts further moves.
3. It displays the game result to the players.

Postconditions: The game concludes, with results visible to players.

Exceptions: None.

Priority: High – Essential for completing each game round.

When available: Within 1 sprint (first iteration).

Frequency of use: Once per game.

Channel to actor: Game result display.

Secondary actors: N/A

Channel to secondary actors: N/A

Open issues: None.

Use Case: Display Game Result

Iteration: 1

Primary Actor: Player

Goal in context: To show the outcome of the game, indicating a winner or a draw.

Preconditions: The game has ended.

Trigger: The game reaches a conclusion (win or draw).

Scenario:

1. The system shows the final game result.
2. It indicates the winning player or a draw.
3. Optionally, players are given the option to start a new game.

Postconditions: Players are informed of the game's outcome.

Exceptions: None.

Priority: Medium – Important for player feedback.

When available: Within 1 sprint (first iteration).

Frequency of use: Once per game.

Channel to actor: Visual display of the result.

Secondary actors: N/A

Channel to secondary actors: N/A

Open issues: None.

Use Case: Switch Player Turns

Iteration: 1

Primary Actor: Player

Goal in context: To alternate between players after each valid move, ensuring fair gameplay.

Preconditions: A player has successfully completed a valid move.

Trigger: A valid move is executed by the current player.

Scenario:

1. The current player completes a valid move.
2. The system identifies the next player in turn.
3. The system updates the active player indicator, signalling whose turn it is.
4. The new active player is allowed to make their move.

Postconditions: The turn alternates to the next player, allowing them to proceed with their move.

Exceptions: If a turn switch error occurs, the system reverts to the previous player for correction.

Priority: High – Ensures balanced and structured gameplay.

When available: Within 1 sprint (first iteration).

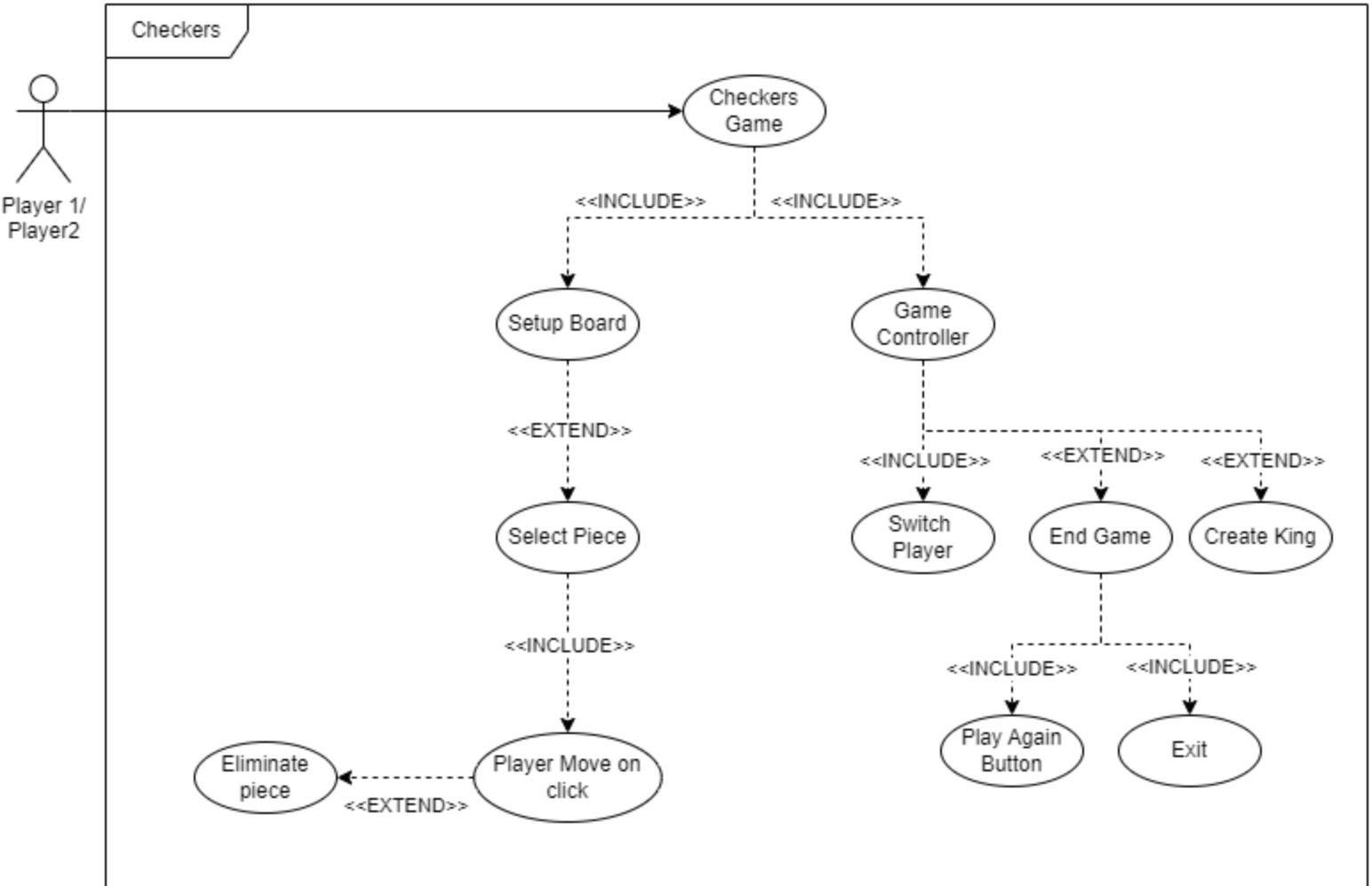
Frequency of use: Every turn, throughout the game.

Channel to actor: Visual or indicator prompt showing the current player. 

Secondary actors: N/A

Channel to secondary actors: N/A

Open issues: None.


Checkers
Player 1/
Player2
Checkers
Game
<<INCLUDE>>
<<INCLUDE>>
Setup Board
Game
Controller
<<EXTEND>>
<<INCLUDE>>
<<EXTEND>>
<<EXTEND>>
Select Piece
Switch
Player
End Game
Create King
<<INCLUDE>>
<<INCLUDE>>
<<INCLUDE>>
Play Again
Button
Exit
Eliminate
piece
Player Move on
click
<<EXTEND>>

Use Case Descriptions Checkers

Use Case: Checkers Game

- Iteration: 1
- Primary Actor: Player 1, Player 2
- Goal in Context: Start and manage a complete game of Checkers between two players.
- Preconditions: None.
- Trigger: A player initiates the game.
- Scenario:
    - The system initializes the game environment and displays the board.
    - The system prepares each player's pieces in starting positions.
    - The game is ready for players to begin taking turns.
- Post Conditions: The game is ready to proceed with players taking turns and making moves.
- Exceptions:
    - If the system fails to initialize, an error message is shown, and the game cannot start.
- Priority: High, as it is essential for gameplay initiation.
- When Available: Within its 1st iteration.
- Frequency of Use: Once per game session.
- Channel to Actor: Player interacts through the game’s user interface.
- Secondary Actors: N/A
- Channel to Secondary Actors: N/A
- Open Issues: None

Use Case: Setup Board

- Iteration: 1
- Primary Actor: System
- Goal in Context: Arrange all pieces on the board for the beginning of the game.
- Preconditions: The Checkers Game has been initiated.
- Trigger: The system initiates board setup as part of game start.
- Scenario:
    - The system places each player's pieces on the designated starting positions.
    - The board is visually updated to show the initial setup.
- Post Conditions: All pieces are correctly positioned on the board for the game to begin.
- Exceptions:
    - If the board setup fails, an error message is displayed, and the game does not start.
- Priority: High, as it is crucial for starting the game.
- When Available: Within its 1st iteration.
- Frequency of Use: Once per game session.
- Channel to Actor: System controls the setup directly. 
- Secondary Actors: N/A
- Channel to Secondary Actors: N/A
- Open Issues: None

Use Case: Game Controller

- Iteration: 1
- Primary Actor: System
- Goal in Context: Manage turn-taking and special game events.
- Preconditions: The game is in progress.
- Trigger: A player ends their turn.
- Scenario:
    - The system ends the current turn.
    - The system enforces any additional game rules (e.g., king creation, elimination).
- Post Conditions: The game continues in accordance with the rules.
- Exceptions:
    - System error in handling turn or rule enforcement.
- Priority: High, essential for game management.
- When Available: Within its 1st iteration.
- Frequency of Use: Continuously throughout the game.
- Channel to Actor: System-managed 
- Secondary Actors: N/A
- Channel to Secondary Actors: N/A
- Open Issues: None

Use Case: Select Piece

Iteration: 1

Primary Actor: Player

Goal in Context: Allows the player to select a piece to move on their turn.

Preconditions: The game must be in progress, and it must be the player's turn.

Trigger: The player clicks on or selects a piece on the board.

Scenario:

1. The player selects a piece by clicking or entering its coordinates
2. The system highlights or marks the selected piece.
3. The system verifies that the selected piece belongs to the current player.

Post Conditions: The piece is highlighted, and the player can now select a destination.

Exceptions:

- The selected piece does not belong to the player, and an error message is shown.

Priority: High priority, as it is essential for gameplay.

When Available: Within its 1st iteration.

Frequency of Use: Multiple times per player turn.

Channel to Actor: Player interacts directly with the board.

Secondary Actors: N/A

Channel to Secondary Actors: N/A

Open Issues: None

Use Case: Move Piece on click

Iteration: 1
Primary Actor: Player

Goal in Context: Allows the player to move a selected piece to a new position on the board.

Preconditions: A piece has been selected by the player, and it is a valid piece to move.

Trigger: The player clicks on or selects a destination square on the board.

Scenario:

1. The player selects a destination for the selected piece.
2. The system checks if the move is valid according to the rules of Checkers. ○ If valid, the piece is moved to the new position.
3. The board is updated to show the new position of the piece.

Post Conditions: The piece has been successfully moved to the new position.

Exceptions:

- The move is invalid, and an error message is displayed.

Priority: High priority, as it is essential for gameplay.
When Available: Within its 1st iteration.
Frequency of Use: Multiple times per game, depending on player moves.
Channel to Actor: Player interacts directly with the board.
Secondary Actors: N/A
Channel to Secondary Actors: N/A
Open Issues: None

Use Case: Eliminate Piece

- **Iteration**: 1
- **Primary Actor**: Player
- **Goal in Context**: Allows the system to remove an opponent’s piece when a capture move is made.
- **Preconditions**: The current player has made a valid capture move.
- **Trigger**: The player completes a capture move.
- **Scenario**:
    - The system identifies the opponent’s piece that is being captured.
    - The captured piece is removed from the board.
    - The board state is updated and displayed to both players.
- **Post Conditions**: The opponent's captured piece is removed from the board.
- **Exceptions**:
    - The system fails to remove the piece due to an error.
- **Priority**: High priority, as it impacts the game's progress and rules.
- **When Available**: Within its 1st iteration.
- **Frequency of Use**: Multiple times per game, depending on capture moves.
- **Channel to Actor**: Game Controller updates the board.
- **Secondary Actors**: System
- **Channel to Secondary Actors**: N/A
- **Open Issues**: None

Use Case: Switch Player

- **Iteration**: 1
- **Primary Actor**: Player
- **Goal in Context**: Allows the system to alternate turns between players.
- **Preconditions**: The current player has completed their turn.
- **Trigger**: The player's turn is over (either after a move or capture).
- **Scenario**:
    - ○ The system verifies that the player's turn is complete.
    - ○ The active player is switched to the other player.
    - ○ The system updates the game state to reflect the new player's turn. ○ The new player is notified that it is now their turn.
- **Post Conditions**: The turn is switched to the other player.
- **Exceptions**:
    - ○ The system fails to switch the player due to an error.
- **Priority**: High priority, as it is critical for turn-based gameplay.
- **When Available**: Within its 1st iteration.
- **Frequency of Use**: Multiple times per game as turns alternate.
- **Channel to Actor**: Game Controller handles turn management.
- **Secondary Actors**: System
- **Channel to Secondary Actors**: N/
- **Open Issues**: None

Use Case: End Game

- **Iteration**: 1
- **Primary Actor**: Player
- **Goal in Context**: Ends the game when a win/loss condition is met or if a player forfeits.
- **Preconditions**: One player has no remaining pieces or legal moves, or a player chooses to forfeit.
- **Trigger**: A win/loss condition is met or a player forfeits.
- **Scenario**:
    - ○ The system checks the board state and verifies if a win/loss condition is met. ○ The game state is updated to GAME_OVER.
    - ○ The system determines and displays the winner.
    - ○ The end-game screen is displayed with options to "Play Again" or "Exit."
- **Post Conditions**: The game ends, and the result is displayed.
- **Exceptions**:
    - ○ The system fails to end the game due to an error.
- **Priority**: High priority, as it is essential for game completion.
- **When Available**: Within its 1st iteration.
- **Frequency of Use**: Once per game.
- **Channel to Actor**: Game Controller manages end-game logic.
- **Secondary Actors**: System
- **Channel to Secondary Actors**: N/A
- **Open Issues**: None

Use Case: Create King

- **Iteration**: 1
- **Primary Actor**: Player
- **Goal in Context**: Promotes a piece to a king when it reaches the opposite end of the board.
- **Preconditions**: A piece reaches the last row on the opponent's side during a move.
- **Trigger**: The player moves a piece to the last row.
- **Scenario**:
  - ○ The system identifies that the piece has reached the opponent's last row. ○ The piece is promoted to a king, with a visual indication.
  - ○ The board state is updated to reflect the promotion.
- **Post Conditions**: The selected piece is promoted to a king with enhanced movement.
- **Exceptions**:
  - ○ The system fails to promote the piece due to an error.
- **Priority**: Medium priority, as it enhances gameplay but is not essential.
- **When Available**: Within its 1st iteration.
- **Frequency of Use**: As needed during gameplay.
- **Channel to Actor**: Game Controller handles promotion.
- **Secondary Actors**: System
- **Channel to Secondary Actors**: N/A
- **Open Issues**: None

Use Case: Play Again Button

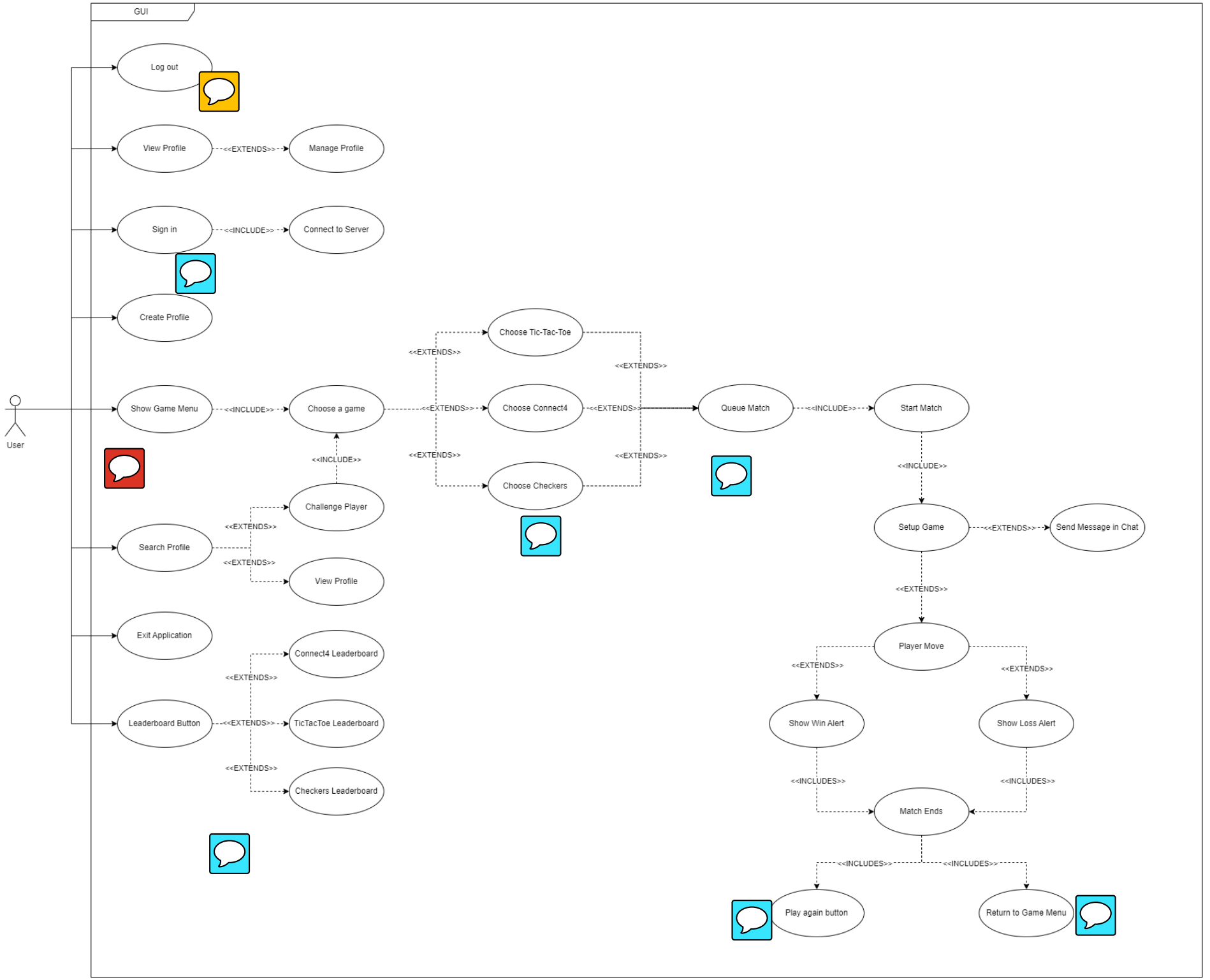
- **Iteration**: 1
- **Primary Actor**: Player
- **Goal in Context**: Allows the player to restart the game with a fresh board after the game has ended.
- **Preconditions**: The game is over, and the "Play Again" button is available on the end-game screen.
- **Trigger**: The player clicks the "Play Again" button.
- **Scenario**:
    - The player selects the "Play Again" button.
    - The system resets the game state and initializes a new board.
    - The board is reinitialized, and the players are notified of the new game start

Post Conditions: A new game begins with the board reset.

- **Exceptions**:
    - The system fails to reset the board due to an error.
- **Priority**: Medium priority, as it enhances user experience.
- **When Available**: Within its 1st iteration.
- **Frequency of Use**: Once per session, if the player wants to replay.
- **Channel to Actor**: User interface for player interaction.
- **Secondary Actors**: N/A
- **Channel to Secondary Actors**: N/A
- **Open Issues**: None

Use Case: Exit

- **Iteration**: 1
- **Primary Actor**: Player
- **Goal in Context**: Allows the player to exit the game after it has ended.
- **Preconditions**: The game is over, and the "Exit" button is available.
- **Trigger**: The player clicks the "Exit" button.
- **Scenario**:
    - The player selects the "Exit" button.
    - The application closes or returns to the main menu.
- **Post Conditions**: The game session is terminated.
- **Exceptions**:
    - The application fails to exit properly.
- **Priority**: Low priority, as it provides a way to end the session.
- **When Available**: Within its 1st iteration.
- **Frequency of Use**: Once per session, if the player wants to end the game.
- **Channel to Actor**: User interface for player interaction.
- **Secondary Actors**: N/A
- **Channel to Secondary Actors**: N/A
- **Open Issues**: None


GUI
Log out
View Profile
<<EXTENDS>>
Manage Profile
Sign in
<<INCLUDE>>
Connect to Server
Create Profile
User
Show Game Menu
<<INCLUDE>>
Choose a game
<<EXTENDS>>
Choose Tic-Tac-Toe
Choose Connect4
Choose Checkers
<<EXTENDS>>
Queue Match
<<INCLUDE>>
Start Match
<<INCLUDE>>
Challenge Player
Search Profile
<<EXTENDS>>
View Profile
Exit Application
Leaderboard Button
<<EXTENDS>>
Connect4 Leaderboard
TicTacToe Leaderboard
Checkers Leaderboard
Setup Game
<<EXTENDS>>
Send Message in Chat
Player Move
Show Win Alert
Show Loss Alert
<<INCLUDES>>
Match Ends
Play again button
Return to Game Menu

Use Case Descriptions (GUI)

Use Case: Log out

Iteration: 1st iteration

Primary actor: User

Goal in context: The user logs out of the system.

Preconditions: The system is powered on, and the user is already logged in with an existing profile.

Trigger: The user clicks on the log out button.

Scenario:

1. The user clicks on the log out button.
2. The user is logged out of the system.

Post Conditions: The user is logged out. 

Exceptions:

- The log out button is unresponsive
- The system encounters an error while initializing
    - Server issues

Priority: High –necessary for log-out.

When Available: Within 1 sprint (1st iteration)

Frequency of Use: At most once per system session.

Channel to actor: Interaction of a click of the log out button with their mouse

Secondary actor: N/A.

Channel to Secondary Actors: N/A

Open issues:

- The setup of the game (any information required) is finished before GUI implementation

Use Case: Manage Profile

Iteration: 2nd iteration

Primary actor: User

Goal in context: The user is given various options for changing their profile.

Preconditions: The game board was set up and is ready for user input.

Trigger: The user has clicked a tile to place their piece. 

Scenario:

1. The user is viewing their own profile.
2. The user clicks on the manage profile button.
3. The user is shown options for changing different aspects of their profile. 

Post Conditions: The user can change different aspects of their profile.

Exceptions:

- The server is down

Priority: Medium-Low – not necessary for the functionality of the games supported by the system.

When Available: Within 2 sprints (2nd iteration)

Frequency of Use: Whenever the user wishes to change their profile.

Channel to actor: Via the system GUI

Secondary actor: N/A

Channel to Secondary Actors: N/A

Open issues: N/A

Use Case: Sign in

Iteration: 1st iteration

Primary actor: User

Goal in context: The user is signed into the system.

Preconditions: The system is powered on, and the user has an existing profile.

Trigger: The user enters a valid username and password combination.

Scenario:



1. The user enters their username.
2. The user enters their password.
3. The user clicks on the sign in button or presses enter.



Post Conditions: A request is made to connect the user to the server.

Exceptions:

- The server is down.

Priority: High – necessary for a user to interact with the system in general.

When Available: Within 1 sprint (1st iteration)

Frequency of Use: Once per user session.

Channel to actor: Via the GUI.

Secondary actor: N/A

Channel to Secondary Actors: N/A

Open issues: N/A

Use Case: Connect to Server

Iteration: 1st iteration

Primary actor: System

Goal in context: Connect the user to the server to enable other interactions.

Preconditions: The system is powered on, and the server is running.

Trigger: A user has attempted to sign in.

Scenario:

1. A user has attempted to sign in.
2. The system connects the user to the server.

Post Conditions: The user is successfully connected to the server.

Exceptions:

- The server is down.

Priority: High – necessary to enable system functionality for the user.

When Available: Within 1 sprint (1st iteration)

Frequency of Use: Once per user session

Channel to actor: Via the network and authentication systems.

Secondary actor: N/A

Channel to Secondary Actors: N/A

Open issues: N/A

Use Case: Create Profile

Iteration: 1st iteration

Primary actor: User

Goal in context: Create a profile for the user.

Preconditions: The system is powered on.

Trigger: The user clicks on the create profile button.

Scenario:

1. The user clicks on the create profile button.
2. The user enters a username. 
3. The user enters a password.
4. The user confirms their entered username and password.
5. The newly created profile is stored in the system.

Post Conditions: The created profile is stored in the system database.

Exceptions:

- The system server is down. 
- The entered username or password are invalid.

Priority: High – necessary for the user to interact with the system in general.

When Available: Within 1 sprint (iteration 1)

Frequency of Use: Whenever the user attempts to create a new profile.

Channel to actor: Via the system GUI.

Secondary actor: N/A

Channel to Secondary Actors: N/A

Open issues: N/A

Use Case: Show Game Menu

Iteration: 1st iteration

Primary actor: User

Goal in context: The user is shown the games supported by the system.

Preconditions: The system is powered on, and the user has signed in.

Trigger: The user clicks on the game menu. 

Scenario:

1. The user clicks on the game menu.
2. The user sees the games supported by the system.

Post Conditions: The games supported by the system are shown to the user, and the user can view.

Exceptions:

- The server is down.
- The game menu is unresponsive.

Priority: High – necessary to allow users to choose which game to play.

When Available: Within 1 sprint (1st iteration)

Frequency of Use: Whenever the user attempts to view the games supported by the system.

Channel to actor: Via the GUI.

Secondary actor: N/A

Channel to Secondary Actors: N/A

Open issues: N/A

Use Case: Choose a Game

Iteration: 1st iteration

Primary actor: User

Goal in context: A game is selected so the user may queue up for a match.

Preconditions: The system is powered on, the user has signed in, and the user is on the game menu.

Trigger: The user clicks on one of the games in the game menu.

Scenario:

1. The user clicks on one of the games in the game menu.
2. The game is selected.
3. Game information and stats are shown and the user may queue for a match.

Post Conditions: The game menu showing the games supported by the system are displayed to the user. 

Exceptions:

- The server is down.

Priority: High – necessary to allow users to queue up for matches and play games.

When Available: Within 1 sprint (1st iteration)

Frequency of Use: Whenever a user clicks on a game.

Channel to actor: Via the game menu.

Secondary actor: N/A

Channel to Secondary Actors: N/A

Open issues: N/A

Use Case: Choose Tic-Tac-Toe

Iteration: 1

Primary actor: User

Goal in context: The user chooses Tic-Tac-Toe from the game menu.

Preconditions: The system is powered on, the user is signed in, and the user is on the game menu.

Trigger: The user clicks on Tic-Tac-Toe from the game menu.

Scenario:

1. The user clicks on Tic-Tac-Toe from the game menu. 
2. Tic-Tac-Toe information and stats are shown to the user, and the option for queueing for a Tic-Tac-Toe match is shown to the user.

Post Conditions: The user has selected Tic-Tac-Toe and is able to queue up for a Tic-Tac-Toe match.

Exceptions:

- The server is down.

Priority: High – necessary for the user to play Tic-Tac-Toe.

When Available: Within 1 sprint (1st iteration)

Frequency of Use: Whenever a user chooses Tic-Tac-Toe from the game menu.

Channel to actor: Via the game menu.

Secondary actor: N/A

Channel to Secondary Actors: N/A

Open issues: N/A

Use Case: Choose Connect4

Iteration: 1

Primary actor: User

Goal in context: The user chooses Tic-Tac-Toe from the game menu.

Preconditions: The system is powered on, the user is signed in, and the user is on the game menu.

Trigger: The user clicks on Connect4 from the game menu.

Scenario:

1. The user clicks on Connect4 from the game menu. 
2. Connect4 information and stats are shown to the user, and the option for queueing for a Connect4 match is shown to the user.

Post Conditions: The user has selected Connect4 and is able to queue up for a Connect4 match.

Exceptions:

- The server is down.

Priority: High – necessary to allow users to play Connect4.

When Available: Within 1 sprint (1st iteration)

Frequency of Use: Whenever the user

Channel to actor: Via the game menu

Secondary actor: N/A

Channel to Secondary Actors: N/A

Open issues: N/A

Use Case: Queue Match

Iteration: 1

Primary actor: User

Goal in context: The user is queued up for a match.

Preconditions: The system is powered on, the user has signed in, and the user has chosen a game.

Trigger: The user clicks on the queue match button.

Scenario:

1. The user clicks on the queue match button. 
2. The system searches for another user queued up for the same game.

Post Conditions: The user is queued up for a match.

Exceptions: 

- The server is down

Priority: High – necessary to allow users to play matches with other users.

When Available: Within 1 sprint (1st iteration)

Frequency of Use: At least once per match.

Channel to actor: Via the GUI

Secondary actor: N/A

Channel to Secondary Actors: N/A

Open issues: N/A

Use Case: Start Match

Iteration: 1

Primary actor: System 

Goal in context: Start a match between two users queued for the same game.

Preconditions: The system is powered on, the servers are running, and at least two users have signed in and are queued for a match on the same game.

Trigger: At least two users are queued up for a match for the same game.

Scenario:

1. At least two users are queued for a match for the same game.
2. The system searches for users queued for the same game.
3. The system matches two users queued for the same game. 
4. The system starts a match between two users queued for the same game.

Post Conditions: A match is started between two users queued for the same game.

Exceptions:

- The server is down.

Priority: High – necessary for users to play games with each other.

When Available: Within 1 sprint (1st iteration)

Frequency of Use: Once per match.

Channel to actor: Via the network system.

Secondary actor: N/A

Channel to Secondary Actors: N/A

Open issues: N/A

Use Case: Search Profile

Iteration: 1

Primary Actor: User

Goal in context: Allows user to search for a profile within the system they want to either view or challenge to a match.

Preconditions: The user has an active session in the main interface where they can access the search function.

Trigger: User initiates a search to find another player profile they would like to view or challenge to a match.

Scenario:

1. User is in the main interface ready to search for another player
2. User enters the name or identifier of the player
3. Search gets processed and retrieves player profile options
4. User selects a player profile from their search results where they can choose to view profile or challenge player

Post-conditions: User lands on selected player profile successfully where they can view or challenge player

Exceptions:

- Unable to find player profile based on search results
- Desired player profile is unavailable
- Unable to search for players if search field is unresponsive

Priority: High. Enhances user experience through being able to interact with other players to challenge and/or view player information.

When available: Within 1st iteration.

Frequency of use: When user wants to search for other players on the platform

Channel to actor: Interaction through search query interface.

Secondary actors: N/A.

Channel to secondary actors: N/A.

Open issues: N/A.

Use Case: Challenge Player

Iteration: 1

Primary Actor: User

Goal in context: Allows user to challenge another player to a match.

Preconditions: The user has an active session in the main interface where they can access the search function, has identified a player, and is ready to initiate challenge to a game.

Trigger: User initiates challenge after selecting a player profile from their search results.

Scenario:

1. User has selected player to challenge based on search results
2. User initiates challenge to selected player

Postconditions: Challenge request has been sent to selected player

Exceptions: 

1. Option to challenge player is unresponsive
2. System fails to send challenge request to desired player

Priority: High. Allows multiplayer gameplay to function with competitive play.

When available: Within 1st iteration.

Frequency of use: Whenever user wants to challenge another player to a match per game session.

Channel to actor: Interaction through challenge player option after landing on desired opponent.

Secondary actors: N/A.

Channel to secondary actors: N/A.

Open issues: N/A.

Use Case: View Profile

Iteration: 1

Primary Actor: User

Goal in context: Allows user to view selected player profile.

Preconditions: The user has an active session in the main interface where they can access the search function and has identified a player they want to view.

Trigger: User selects a player profile from search results they want to view details of.

Scenario: 

1. User has selected a player from search results
2. User initiates profile overview for desired player profile

Postconditions: User is able to view selected player's profile successfully where they can access match history, wins/losses, etc…

Exceptions:

1. Unable to find player profile based on search results
2. System fails to retrieve profile details regarding desired player profile

Priority: High. Enhances user experience through being able to view other user statistics.

When available: Within 1st iteration.

Frequency of use: Whenever user wants to view other player profiles.

Channel to actor: Interaction through view player profile option upon selecting player they want to view.

Secondary actors: N/A.

Channel to secondary actors: N/A.

Open issues: N/A.

Use Case: Exit Application

Iteration: 1

Primary Actor: User

Goal in context: Allows user to exit out of the main application.

Preconditions: The user is in the main interface and wishes to exit out of the application.

Trigger: User presses the “Exit” button to exit session.

Scenario:

1. User is in the main menu interface
2. User selects the “Exit” button from the application menu to quit session

Postconditions: Application is closed and the user is out of the application.

Exceptions:



1. “Exit” button fails to exit user out and their application remains open

Priority: High. Users should be able to exit out of the application whenever they desire.

When available: Within 1st iteration.

Frequency of use: Once per active user session.

Channel to actor: “Exit” button in the main menu interface.

Secondary actors: N/A.

Channel to secondary actors: N/A.

Open issues: N/A.

Use Case: Leaderboard Button

Iteration: 1

Primary Actor: User

Goal in context: Allows user to view the leadership board for various games.

Preconditions: The user is in the main interface and wants to view the leadership board for various games.

Trigger: User presses the “Leaderboard” button

Scenario:

1. User is in the main menu interface
2. User selects the “Leaderboard” button from the menu

Postconditions: Display of leaderboards for Tic-Tac-Toe, Checkers, and Connect4 are available.

Exceptions: 

1. “Leadership” button is unresponsive
2. Leaderboard button fails to retrieve data for leadership statistics of various games

Priority: High. Access to leaderboard is crucial for user engagement and competitive play.

When available: Within 1st iteration.

Frequency of use: Whenever user wants to check out leaderboard options.

Channel to actor: “Leaderboard” button in main menu interface.

Secondary actors: N/A.

Channel to secondary actors: N/A.

Open issues: N/A.

Use Case: Connect4 Leaderboard

Iteration: 1

Primary Actor: User

Goal in context: Allows user to view the leadership board for Connect 4.

Preconditions: The user is in the main interface and wants to view the leadership board for Connect 4.

Trigger: User presses the “Connect4 Leaderboard” button

Scenario:

1. User is in the main menu interface 
3. User selects the “Leaderboard” button from the menu
2. User selects the “Connect4 Leaderboard” button 

Postconditions: Display of leaderboard for Connect 4 made available for user.

Exceptions: 

1. “Connect4 Leadership” button is unresponsive
2. System fails to retrieve information regarding Connect4 leaderboard data

Priority: High. Access to leaderboard is crucial for user engagement and competitive play.

When available: Within 1st iteration.

Frequency of use: Whenever user wants to view leadership information for Connect4

Channel to actor: “Connect4 Leaderboard” button after selecting “Leaderboard” option

Secondary actors: N/A.

Channel to secondary actors: N/A.

Open issues: N/A.

Use Case: TicTacToe Leaderboard

Iteration: 1

Primary Actor: User

Goal in context: Allows user to view the leadership board for Tic-Tac-Toe game.

Preconditions: The user is in the main interface and wants to view the leadership board for Tic-Tac-Toe.

Trigger: User presses the “Tic-Tac-Toe Leaderboard” button

Scenario:

1. User is in the main menu interface 
2. User selects the “Leaderboard” button from the menu
3. User selects the “Tic-Tac-Toe Leaderboard” button 

Postconditions: Display of leaderboard for Tic-Tac-Toe made available for user.

Exceptions: 

1. “Tic-Tac-Toe Leadership” button is unresponsive
2. System fails to retrieve information regarding Tic-Tac-Toe leaderboard data

Priority: High. Access to leaderboard is crucial for user engagement and competitive play.

When available: Within 1st iteration.

Frequency of use: Whenever user wants to view leadership information for Tic-Tac-Toe

Channel to actor: “Tic-Tac-Toe Leaderboard” button after selecting “Leaderboard” option

Secondary actors: N/A.

Channel to secondary actors: N/A.

Open issues: N/A.

Use Case: Checkers Leaderboard

Iteration: 1

Primary Actor: User

Goal in context: Allows user to view the leadership board for Checkers game.

Preconditions: The user is in the main interface and wants to view the leadership board for Checkers.

Trigger: User presses the “Checkers Leaderboard” button

Scenario:

1. User is in the main menu interface 
2. User selects the “Leaderboard” button from the menu
3. User selects the “Checkers Leaderboard” button 

Postconditions: Display of leaderboard for Checkers made available for user. 

Exceptions:

1. “Checkers Leaderboard” button is unresponsive
2. System fails to retrieve information regarding Checkers leaderboard data

Priority: High. Access to leaderboard is crucial for user engagement and competitive play.

When available: Within 1st iteration.

Frequency of use: Whenever user wants to view leadership information for Checkers.

Channel to actor: “Checkers Leaderboard” button after selecting “Leaderboard” option

Secondary actors: N/A.

Channel to secondary actors: N/A.

Open issues: N/A.

Use Case: Choose Checkers

Iteration: 1

Primary Actor: User

Goal in context: Allows user to play Checkers game if that is the game they want to play.

Preconditions: The user is in the main interface, they are on the interface to choose a game, Checkers is available to play for the user.

Trigger: User presses the “Choose Checkers” button

Scenario:

1. User is in the game menu
2. User selects the “Choose Checkers” option from the list of available games to play



Postconditions: Display of Checkers grid will be available for user to play.

Exceptions:



1. Checkers game fails to load due to technical issues
2. Interface is unresponsive when Checkers is selected

Priority: High. Users should be able to choose Checkers as an available game option.

When available: Within 1st iteration.

Frequency of use: Whenever user wants to play Checkers.

Channel to actor: “Choose Checkers” option as one of the games available for the user to select in the game menu.

Secondary actors: N/A.

Channel to secondary actors: N/A.

Open issues: N/A.

Use Case: Setup Game

Iteration: 1

Primary Actor: User

Goal in context: Sets up game necessary for user to engage in.

Preconditions: The user has selected the game they want to play from the options available and has started a match

Trigger: User has an active game match session. 

Scenario:

1. User has started a game session after matchmaking finished
2. The interface displays and loads the game environment
3. User is presented with the game board they chose and resources to play

Postconditions: Game is fully set-up for user to engage in.

Exceptions:

1. Game fails to set-up properly through configuration errors
2. Interface for game mode is dysfunctional

Priority: High. Game needs to be set-up in order for gameplay to begin.

When available: Within 1st iteration.

Frequency of use: Once per active game session

Channel to actor: Through the selective game environment of their preferred game.

Secondary actors: N/A.

Channel to secondary actors: N/A.

Open issues: N/A.

Use Case: Send Message in Chat

Iteration: 1

Primary Actor: User

Goal in context: Allows communication between users in-game.

Preconditions: User has entered the game environment they have chosen after starting a match and game has been set-up successfully with the chat feature.

Trigger: User enters a message into the chat field and sends it to the entire game room.

Scenario:

1. User has entered the game session after matchmaking is complete
2. The game environment is set-up including the chat feature
3. User types a message into the chat field 
4. User presses “Send” button
5. The message is displayed for all players to see inside the game session

Exceptions:

1. Chat feature fails to display from configuration issues
2. Chat feature is unresponsive preventing messages to be sent
3. Messages are not visible to everyone

Postconditions: Message is sent and visible to everyone engaged in the space

Priority: Medium. Non-essential but allows in-game communication between players.

When available: Within 1st iteration.

Frequency of use: Multiple times within one game session.

Channel to actor: Chat field within chosen game session.

Secondary actors: N/A.

Channel to secondary actors: N/A.

Open issues: N/A.

Use Case: Player Move

Iteration: 1

Primary Actor: User

Goal in context: Allows player to take their turn

Preconditions: Game environment has been properly set-up with players in session

Trigger: User initiates a move when it is their turn

Scenario:

1. User has entered the game session after matchmaking is complete
2. The game environment is set-up properly
3. It has indicated that it is the user's turn
4. User makes their move (ex. moving a piece, selecting a grid space)

Exceptions:



1. Move failed to process
2. User turn-taking is dysfunctional
3. Interface is unresponsive

Postconditions: Player's move is processed and the game state is updated accordingly

Priority: High. Allows active gameplay to happen.

When available: Within 1st iteration.

Frequency of use: Once per turn iteration.

Channel to actor: Game environment interface where moves can be made

Secondary actors: N/A.

Channel to secondary actors: N/A.

Open issues: N/A.

Use Case: Show Win Alert

Iteration: 1

Primary Actor: User

Goal in context: Alerts user if a player has won the game (either user or opponent)

Preconditions: User is in game session and a win condition is met by either user or opponent

Trigger: Game detects that a player has met the win condition after a move

Scenario:

1. User is in-game session
2. Game is set-up
3. User makes move when it is their turn
4. Game logic processes the move to check for win condition
5. If win condition met, then a win alert is displayed and the match is ended

Exceptions:

1. Win condition fails to process
2. Win alert fails to display on screen 

Postconditions: User is made aware of win condition via alert and the match is concluded

Priority: High. Should indicate to user if a win condition is met to end the match

When available: Within 1st iteration.

Frequency of use: Once per game session when win condition is detected

Channel to actor: A win alert is displayed on the screen for user to view

Secondary actors: N/A.

Channel to secondary actors: N/A.

Open issues: N/A.

Use Case: Show Loss Alert

Iteration: 1

Primary Actor: User

Goal in context: Alerts user if a player has lost the game (either user or opponent)

Preconditions: User is in game session and a loss condition is met by either user or opponent (the other player has won)

Trigger: Game detects that a player has met the win condition after a move

Scenario:

1. User is in-game session
2. Game is set-up
3. User makes move when it is their turn
4. Game logic processes the move to check for win condition
5. If win condition met for either player, then the other player receives a loss

Exceptions:

1. Win condition fails to process
2. Loss alert fails to display on screen 

Postconditions: User is made aware of loss via alert and the match is concluded

Priority: High. Should indicate to user if a loss is met to end the match

When available: Within 1st iteration.

Frequency of use: Once per game session when win condition is detected

Channel to actor: A loss alert is displayed on the screen for user to view

Secondary actors: N/A.

Channel to secondary actors: N/A.

Open issues: N/A.

Use Case: Match Ends

Iteration: 1

Primary Actor: User

Goal in context: Shows to user that a match is concluded after an outcome is met

Preconditions: User is in game session and a win condition has been met by either the user or the opponent

Trigger: Game detects that a game has ended via player win

Scenario:

1. User is in a game session
2. Game is set-up
3. Game logic has processed that a win condition has been met by either player after a turn
4. The system concludes the match, showing appropriate alert to the user

Exceptions:

1. Win condition fails to process
2. Loss alert fails to display on screen

Postconditions: Match has ended and user can play again or ret  to the game menu

Priority: High. Match needs to conclude after outcome reached and user can select next action

When available: Within 1st iteration.

Frequency of use: Once per game session when win condition is detected

Channel to actor: Match conclusion is displayed to the user on the screen

Secondary actors: N/A.

Channel to secondary actors: N/A.

Open issues: N/A.

Use Case: Play Again Button

Iteration: 1

Primary Actor: User

Goal in context: Allows user to play again if they so choose to after a match has ended

Preconditions: A match has concluded based on either player reaching the win condition

Trigger: User presses the “Play Again” button after a match has finishe

Scenario:

1. The system detects a win condition has been met
2. The match concludes for the user
3. The user is provided with the option to play again

Exceptions:

1. “Play Again” button is unresponsive
2. Failure to reset the game environment

Postconditions: User is brought back to game set-up screen to open a new match

Priority: High. Provides user with intuitive option to continue playing after a match has ended without going back to the main menu as a shortcut

When available: Within 1st iteration.

Frequency of use: Once per game session after match has concluded

Channel to actor: Through the “Play Again” button in the interface

Secondary actors: N/A.

Channel to secondary actors: N/A.

Open issues: N/A.

Use Case: Return to Game Menu

Iteration: 1

Primary Actor: User

Goal in context: Allows user to return back to the game menu after a match has concluded to access other options

Preconditions: A match has concluded based on either player reaching the win condition

Trigger: User indicates they want to go back to the game menu by selecting "Return to Game Menu"

Scenario:

1. The system processes that a win condition has been met
2. The match is concluded with displays of win/loss to user
3. User is presented with the option to return to the game menu

Exceptions:

1. Button is unresponsive
2. Failure to load the main menu upon input

Postconditions: User is taken back to the game menu upon choosing to return

Priority: High. Provides user with a way to exit a concluded match and to navigate back to the main menu.

When available: Within 1st iteration.

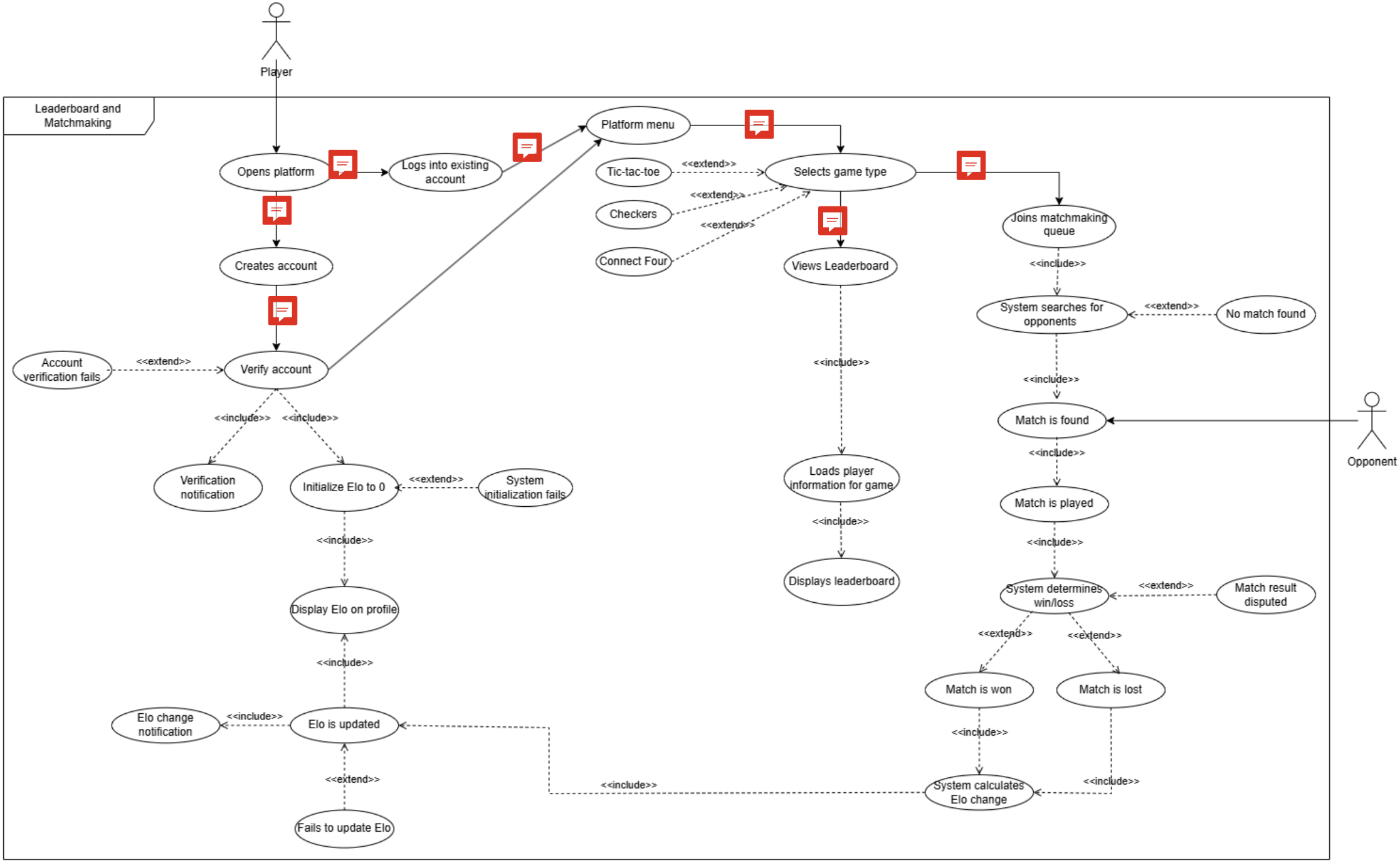
Frequency of use: Once per game session after match has concluded

Channel to actor: "Return to Game Menu" button available for user to select after a match has concluded

Secondary actors: N/A.

Channel to secondary actors: N/A.

Open issues: N/A.


Player
Leaderboard and Matchmaking
Opens platform
Logs into existing account
Platform menu
Creates account
Verify account
Account verification fails
<<extend>>
<<include>>
<<include>>
Verification notification
Initialize Elo to 0
<<extend>>
System initialization fails
<<include>>
Display Elo on profile
<<include>>
Elo is updated
Elo change notification
<<include>>
<<extend>>
Fails to update Elo
Tic-tac-toe
Checkers
Connect Four
<<extend>>
<<extend>>
<<extend>>
Selects game type
Views Leaderboard
<<include>>
Loads player information for game
<<include>>
Displays leaderboard
Joins matchmaking queue
<<include>>
System searches for opponents
<<extend>>
No match found
<<include>>
Match is found
Opponent
<<include>>
Match is played
<<include>>
System determines win/loss
<<extend>>
Match result disputed
<<extend>>
<<extend>>
Match is won
Match is lost
<<include>>
<<include>>
System calculates Elo change
<<include>>

Use Case: Initialize Player Elo

Iteration: 1

Primary Actor: Player

Goal In Context: Initialize the elo to 0 for all new player accounts

Preconditions:
- The player has successfully registered and verified their account

Trigger: User finishes creating account

Scenario:
1. The player completes the registration process.
2. The system verifies the player's account.
3. The system initializes the player's elo to 0.
4. The player is notified that their account setup is complete and the elo rating is displayed on the player's profile**.**

Post-conditions:
- Account is initialized with elo of 0
- Player receives confirmation of account setup and is ready to join matchmaking queue

Exceptions:
- Account verification fails
- System initialization fails

Priority: High, basic game function

When available: First increment or iteration

Frequency of use: Each time a new player registers

Channel to actor: Game interface

Secondary actors: N/A

Channel to secondary actors: N/A

Open issues:
- How are we going to handle duplicate accounts?
- How can we ensure elo initialization is tamper-proof?

Use Case: Matchmaking Queue

Iteration: 1

Primary Actor: Player

Goal In Context: Allow a player to join a matchmaking queue and be paired with an opponent of similar elo rating

Preconditions:
- The player has logged into the multiplayer platform
- The player has a valid player profile and elo rating

Trigger: The player selects the “Join Matchmaking Queue” option

Scenario:
1. The player selects the matchmaking option.
2. The system searches the matchmaking queue for opponents with similar elo ratings.
3. Once a match is found, the system notifies both players.
4. The game session is initialized.

Post-conditions:
- The player is paired with an opponent and starts a game

Exceptions:
- No match is found, and the player remains in the queue until an appropriate opponent joins

Priority: High, core game mechanic

When available: First increment or iteration

Frequency of use: Each time a player wants to play a match

Channel to actor: Game interface

Secondary actors: Opponent Player

Channel to secondary actors: Game interface, matchmaking system

Open issues:
- How long to keep players waiting in the queue if no suitable opponent is found?
- If no suitable opponent is found, how long till the player is assigned someone that is out of their elo rating range?

Use Case: Update Player Elo

Iteration: 1

Primary Actor: Player

Goal In Context: Update the player's elo based on the outcome of a match

Preconditions:
- A game has concluded, and the winner and loser are determined
- Both players have valid elo ratings

Trigger: Match ends and results are recorded

Scenario:
1. The game ends, and the system determines the winner and loser.
2. The system calculates the elo change based on the match outcome.
3. The player's elo is updated accordingly.
4. The player is notified of their change in elo, which is visible on their profiles.

Post-conditions:
- Player's elo rating is updated
- Player receives notification of the change in elo

Exceptions:
- Match result is disputed
- System fails to update the elo ratings

Priority: High, basic game function

When available: First increment or iteration

Frequency of use: After each match

Channel to actor: Game interface

Secondary actors: Opponent player

Channel to secondary actors: Game interface

Open issues:
- How can we ensure accurate and fair elo calculations?

Use Case: View Leaderboard

Iteration: 1

Primary Actor: Player

Goal In Context: View the leaderboard to see the top-ranked players based on elo rating for each game type

Preconditions:
- Players have played games and have valid elo ratings
- Leaderboard data is updated regularly

Trigger: User selects "View Leaderboard" from the menu

Scenario:
1. The user selects the "View Leaderboard" option.
2. The system retrieves the list of players sorted by their elo ratings, highest to lowest.
3. The leaderboard is displayed for the selected game type.
4. The user views the leaderboard rankings.

Post-conditions:
- The leaderboard is shown with updated player rankings

Exceptions:
- Leaderboard data fails to load

Priority: Medium, players are not required to view leaderboard

When available: First increment or iteration

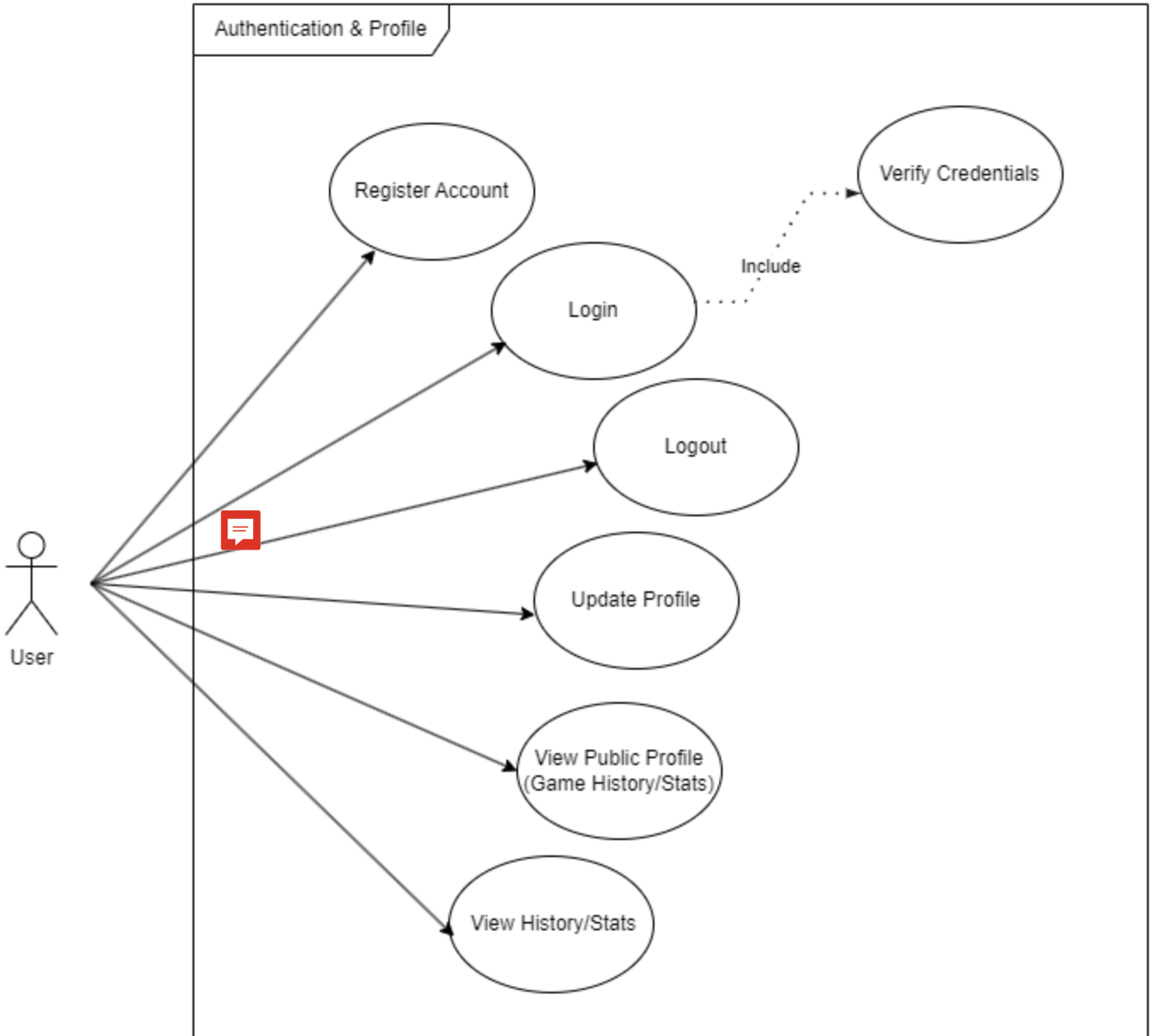
Frequency of use: Regularly by players

Channel to actor: Game interface

Secondary actors: N/A

Channel to secondary actors: N/A

Open issues:
- How many players should the leaderboard display?


Authentication & Profile
Register Account
Verify Credentials
Include
Login
Logout
Update Profile
View Public Profile
(Game History/Stats)
View History/Stats
User

Use Case: User Registration

Iteration: 1

Primary Actor: User

Goal in Context: Register a new account.

Preconditions: User is on the User Registration GUI

Trigger: User clicks on the “Register” button

Scenario:

1. The user provides username and password, email
2. System makes sure username is unique
3. System stores this new username and password in the database.

Postcondition: User Registration is complete, an account has been created.

Exceptions:

- Username/email (optional) already exists

Priority: High. As it’s crucial to have an account to play games.

When available: Increment 1

Frequency of use: High

Channel to actor: GUI

Secondary actors: Database

Channel to secondary actors: Database/API?

Open issues:

- Should emails be mandatory in the process of user registration?

Use Case: User Login

Iteration: 1

Primary Actor: User.

Goal in Context: Login and access the application.

Preconditions: User account exists

Trigger: User clicks the Login button.

Scenario:

1. User enters username and password
2. System Verifies Credentials (See Verify Credentials Usecase)
3. User has access to the platform.

Postcondition: User is logged in.

Exceptions:

- Incorrect password and/or username doesn't exist. (See Verify Credentials Use Case)

Priority: High

When available: Increment 1

Frequency of use: Frequent

Channel to actor: GUI

Secondary actors: Database

Channel to secondary actors: Database

Open issues: (none)

Use Case: Verify Credentials

Iteration: 1

Primary Actor: Database

Goal in Context: Verify account exists and login information is correct

Preconditions: User has entered username and password

Trigger: User clicks the Login button.

Scenario: 

1. Username is located in database
4. System confirms password entered matches password associated with the username
5. User is granted access

Postcondition: User's credentials have been verified

Exceptions:

- Username doesn't exist, login denied
- Password doesn't match, login denied

Priority: High

When available: Increment 1

Frequency of use: Frequent.

Channel to actor: GUI

Secondary actors: (none)

Channel to secondary actors: (none)

Open issues: (none)

Use Case: View Public Profile

Iteration: 1

Primary Actor: Registered User

Goal in Context: Be able to View profile details and game history (Wins and losses).

Preconditions: User is logged in.

Trigger: User clicks on “View Profile”

Scenario:

1. Users are able to view their profile.
2. System shows the game history (Wins, losses, games played, player statistics) and other relevant information

Postcondition: Profile details and stats are displayed.

Exceptions:

- GUI issues
- Data issues.

Priority: Medium

When available: Iteration 1

Frequency of use: Frequent

Channel to actor: GUI

Secondary actors: Database

Channel to secondary actors: Database

Open issues:

- How much can the user see?

Use Case: Logout

Iteration: 1

Primary Actor: Registered User

Goal in Context: Log out

Preconditions: User is logged in.

Trigger: User clicks Logout button

Scenario:

1. User clicks the Logout button
2. System logs the user out
3. User is redirected to login page

Postcondition: User is logged out

Exceptions:

- Data issues

Priority: High

When available: Iteration 1

Frequency of use: Frequent

Channel to actor: GUI

Secondary actors: Data base?

Channel to secondary actors: Data base?

Open issues:

- Should there be a confirmation popup before logging the user out?

‘

Use Case: Update Profile

Iteration: 2

Primary Actor: Registered User

Goal in Context: Update profile information

Preconditions: User is logged in

Trigger: User clicks on update profile option

Scenario:

1. Profile details are visible to the user
2. User can select which details they would like to update
3. User can update their profile information details
4. User clicks save

Postcondition: Profile updates are saved

Exceptions:

- User clicks cancel/exit and profile updates are not saved

Priority: Medium

When available: Iteration 2

Frequency of use: Occasionally

Channel to actor: GUI

Secondary actors: Data base?

Channel to secondary actors: Data base?

Open issues:

- What profile details are visible?

Use Case: View Public Game History/Statistics

Iteration: 1

Primary Actor: Registered User

Goal in Context: View public game history/statistics

Preconditions: User is logged in

Trigger: User clicks Player Hub option

Scenario:

1. Current leaderboard is displayed on screen
2. List of players is displayed on screen
3. User can select player profile to view
4. Public player information, status, game history and ranking is displayed on screen

Postcondition: Public player information is displayed

Exceptions: (none)

Priority: Medium

When available: Iteration 2

Frequency of use: Frequent

Channel to actor: GUI

Secondary actors: Data base

Channel to secondary actors: Data base

Open issues:

- How much profile information is available to other players?